Bachelor Thesis, 15 ECTS — Machine Learning and Data Science

A Machine-Learning Model of Inter-Keystroke Intervals for Evaluating Typing Systems

[redacted]

KU-ID: [redacted]

Main supervisor: [redacted]

Co-supervisor: [redacted]

University of Copenhagen, DIKU

2026-06-12

Abstract

Typing speed has traditionally been estimated with simple heuristic models. This project replaces them with machine learning models trained on ≈4.8M keystrokes of typing data [1] to predict Inter-Keystroke Intervals (IKI). Multi-Layer Perceptron and LightGBM models achieve a mean absolute error (MAE) of 0.57 on a QWERTY validation set. These models generalize to the Dvorak layout with an MAE of 0.59 despite not being trained on it, the first zero-shot evaluation of a typing-speed model on a different layout. The models are used to evaluate four typing system optimizations. A one-shot shift key (tapped before a letter to capitalize it) yields at least a 0.71% gain, and a repeat key (tapped after a key to repeat it) a 0.26% gain, each displacing the semicolon. Optimized auto-expanding abbreviation dictionaries increase speed by up to 13% for a 160-entry dictionary, assuming a typist recalls each mapping instantly. The model indicates a 0.7% speed benefit for Dvorak over QWERTY. While the LightGBM model exhibits no statistically significant change in systematic error (bias shift) between layouts, the lack of data for layouts other than QWERTY and Dvorak limits the ability to evaluate arbitrary keyboard layouts. More non-QWERTY typing data would most improve the models.

Contents

1 Introduction

Faster typing can increase productivity. This project builds a machine learning model to predict keyboard typing speed, providing a way to evaluate text input systems.

Many hypotheses have been proposed about what affects typing speed. Some are about typing system design: alternative layouts such as Dvorak and Colemak, abbreviation expansion, chording, dead keys, one-shot modifiers, layer keys, and repeat keys. Others are about specific motor patterns: same-finger bigrams, finger travel distance, hand alternation, and rolling movements.

Testing these hypotheses empirically is difficult because it is hard to control for familiarity. Almost everyone is already familiar with standard QWERTY without advanced features such as repeat keys or one-shot modifiers. Any trial involving a novel layout or input mechanism confounds the system's intrinsic effect with the practice needed to learn it. Learning times vary considerably by system, but can take hundreds of hours [2], [3].

A predictive model bypasses this learning confound. Trained on real keystroke data, the model evaluates typing speed on novel sequences without requiring any participant to type them. The model targets skilled, fast typists, because the systems it evaluates, such as abbreviation expansion, are designed for people who want to type fast. Such a model has several uses:

Testing the above hypotheses. The model estimates the speed benefit of typing system features, without requiring participants to learn them.

Layout optimization. A model of what determines typing speed could enable optimizing keyboard layouts for speed, without recruiting participants.

Abbreviation expansion. Text expansion systems let users type a short sequence that automatically expands to a full word or phrase.

Research normalization. Experiments often compare typing speed across different text. Some text is simply harder to type than other text. The model measures this difficulty, so researchers can control for it [4].

The project has two parts. The first builds and validates the predictive model. The second applies it in four studies. Each study tests one way to type faster. These come from the designs listed above: two new keys, an abbreviation dictionary, and a different keyboard layout. Each is meant to speed up typing, but its real effect has been hard to measure. The model can now estimate it:

1. Does a one-shot shift key — tapped once to shift only the next keystroke, instead of being held down — reduce typing time?
2. Does a repeat key — a key that re-emits the previous keystroke — reduce typing time?
3. Does an abbreviation dictionary reduce typing time, and by how much?
4. Is Dvorak faster than QWERTY?

The project also examines whether the model can guide layout optimization.

2 Background

2.1 Typing speed metrics

Typing speed has two common units. Words per minute (WPM) aggregates over a passage. Inter-Keystroke Interval (IKI) measures the time between two consecutive keypresses, typically in milliseconds. This project works with IKI.

2.2 Prior models

Early models of typing speed used for layout evaluation relied on heuristics. Dvorak's original work [5] motivated layout design primarily through reduced finger travel and hand alternation. Later models such as CARPALX [6] formalised this into weighted effort scores combining row penalties, finger usage, and bigram costs. These models share a common limitation: their weights are hand-assigned, not learned from typing data. They also typically target both speed and ergonomics; this project addresses speed only.

A small body of work used measured typing data.

Kinkead [7] conducted an early computational evaluation. They captured 115 000 keystrokes from 22 typists during standardised speed typing tests on a QWERTY layout, and classified each keystroke by its relation to the previous one (opposite hand, same hand different finger, same finger, repeated key). The analysis identified alternate-hand strokes as fastest and same-finger strokes as slowest. They used mean class times and English digram frequencies to estimate Dvorak speed without recruiting Dvorak typists. The model predicted Dvorak as 2.6% faster than QWERTY and an optimal layout as 7.6% faster.

Hiraga et al. [8] took a regression-based approach. From 302 392 keystrokes typed by a single professional typist on QWERTY, they fit a multivariate linear model predicting IKI from hand alternation, row transition, finger distance, and keystroke frequency. Hand alternation was the dominant factor, giving a ≈40 ms advantage for alternate-hand strokes, consistent with Kinkead's findings.

Işeri and Ekşioğlu [9] measured digraph timings directly through repeated tapping rather than real typing, for the purpose of designing an optimized Turkish layout. Seven participants tapped each of 241 same-hand digraph pairs in isolation at maximum speed for two minutes per pair, yielding per-digraph timing rates. Alternating-hand digraph costs were not measured directly; they were derived from the same-hand results using a scaling factor from Salthouse [10], making that part a heuristic estimate rather than an empirical measurement. An ANOVA identified the column (which finger), row, hand, and period as significant factors, with finger column carrying the most explanatory power. Combining these empirical digraph costs with Turkish digraph frequencies in a quadratic assignment problem, they derived an optimized layout that outperformed the existing Turkish layouts on both the cost objective and Dvorak's heuristic criteria.

Williams et al. [4] took the largest-scale data-driven approach to date. They trained on real typing data from the 136M Keystrokes dataset [1], where 168 000 participants each typed 15 sentences from a pool of 1 493 English items. For each sentence, individual typing speeds were z-scored within participant and averaged across participants, producing a sentence-level "typability" score. Random forest regression selected eight predictors from 30 candidates spanning linguistic, layout, and biomechanical attributes (most importantly the proportion of lowercase characters, total keystrokes, and syllables per word), which were then combined in a multiple linear regression model. The model explained 74% of typability variance in training and 68% on a held-out test set.

2.3 Limitations of existing approaches

Table 1 summarises which limitations apply to each prior approach.

Limitation	CARPALX	Kinkead	Hiraga	Işeri	Williams	This work
Sentence-level granularity	✗				✗	
Linear model	✗	✗	✗	✗	✗	
Arbitrary weights	✗			(✗)		
Hand-designed features	✗	✗	✗	✗	✗	(✗)
No anticipation	(✗)	✗	✗	✗		
Linguistic confounding	n/a	✗	✗		✗	(✗)
Small sample	n/a	✗	✗	✗		(✗)
Isolated tapping	n/a			✗		
QWERTY only		✗	✗		✗	(✗)

Table 1: Limitations of prior approaches. ✗ = present; (✗) = partially present; blank = absent; n/a = not applicable.

Granularity. The Typability Index predicts at sentence level, producing one score regardless of how many keystrokes a sentence contains. IKI is necessary to capture key-to-key transitions directly.

Linearity. All previous models combine features linearly. Linear models cannot easily capture interactions such as a longer reach being especially bad when performed using pinky fingers.

Arbitrary heuristic weights. Models like CARPALX and Dvorak's original criteria combine factors using weights chosen by researchers rather than learned from data. The relative cost of, say, a pinky reach versus an index stretch has not been empirically grounded.

Hand-designed features. All data-driven predecessors rely on hand-designed features (finger travel, syllable counts, word frequency). Whether these capture the relevant motor signal completely is untested; a deep learning model trained on raw data could find patterns no one has labeled.

Anticipation. Salthouse [10] argues that typing is chunked: typists process upcoming characters in parallel with executing the current keystroke. If this is correct, IKI depends not only on the preceding key but also on what follows. The heuristic and bigram-cost models ignore future context entirely.

Linguistic confounding. Kinkead's and Hiraga et al.'s keystrokes came from real QWERTY typing tests, where linguistically common bigrams may appear fast because typists have practised them more, not because the underlying movement is easier. This conflates biomechanical cost with practice.

Small samples. Kinkead used 22 typists, Hiraga et al. a single typist, and Işeri and Ekşioğlu seven participants. Estimates from such small samples may not generalise.

Isolated tapping. Işeri and Ekşioğlu avoided the linguistic confound by having participants tap each digraph in isolation, but the resulting measure reflects motor capacity for one repeated motion rather than timing within real text, where surrounding keys influence each transition.

Generalisation. All data-driven predecessors were trained and evaluated on QWERTY data with standard English prose. It remains untested whether models transfer to alternative layouts (Dvorak, Colemak).

Training a model that avoids these limitations is not straightforward. Keystroke datasets are almost exclusively QWERTY-based, so there is a risk of learning QWERTY-specific patterns rather than general motor principles. Linguistic and motor cost are entangled: common sequences may be fast because they are practised, not because they are physically easier. IKI data is also noisy; the same transition typed by the same person in the same context shows variance.

2.4 This project's response

Training on per-keystroke IKI from the 136M Keystrokes dataset [1] (168 000 participants) resolves the granularity issue and some sample-size issues. A windowed input spanning future keys adds anticipatory context. Non-linear architectures (gradient-boosted trees and MLPs) capture interactions that linear models miss. Learned weights replace hand-assigned heuristics. Zero-shot evaluation on Dvorak tests cross-layout generalization directly. The linguistic confound is only partly resolved. The dataset is real QWERTY typing, so familiarity and motor cost stay entangled. Explicit linguistic features (word frequency, bigram frequency) let the model assign some of the speed to familiarity rather than motor cost, but cannot separate the two fully.

3 Methodology

The goal is to train ML models on QWERTY data that predict inter-keystroke intervals on Dvorak data.

The methodology is a pipeline from raw keystroke logs to evaluated models. Four stages produce the modeling dataset: selection picks the typists and keystrokes, preparation cleans them, enrichment computes the features, and splitting divides the data into training and validation sets. The model architectures are introduced next (Section 3.5), as context for the modeling stages that follow. Feature selection, hyperparameter optimization, and final training produce the models. A last stage defines the metrics and statistical tests used to evaluate them.

All code is available at `github.com/jan-mate/typing-model`.

3.1 Data

This project uses two datasets. The 136M Keystrokes dataset provides the keystroke timings that train and evaluate the IKI model. A combined Reddit and Wikipedia corpus aims to provide realistic text for frequency features and for evaluating typing systems.

3.1.1 136M Keystrokes dataset

Training uses the Aalto 136M Keystrokes dataset [1]: over 136 million keystrokes from 168 594 participants performing transcription tasks. Each participant transcribed 15 sentences while the system logged key-press and key-release timestamps. From the original corpora, the dataset's creators chose random sentences containing at least 3 words and at most 70 characters, fewer than five numerical symbols, and only simple punctuation marks (`,`, `.`, `!`, `?`, `'`). The dataset's scale and coverage of both QWERTY and Dvorak typing motivate its choice for this project. QWERTY data trains and validates the models; the pipeline holds out the smaller pool of Dvorak data to test cross-layout generalisation. The two layouts are heavily imbalanced: of the 168 594 participants, 165 324 type on QWERTY and only 209 on Dvorak. Visual representations of QWERTY, Dvorak, and the other layouts used for evaluation are provided in Appendix A.1. Table 2 shows the format of a processed sequence.

Key	p	o	p	p	y
IKI (ms)	NaN	170	150	180	110

Table 2: A processed keystroke sequence: each key with the interval since the previous key.

Cross-layout evaluation requires distinguishing a typed character (keycode) from its physical position on the board (keyslot). The model covers 42 keyslots. These positions cover the 26 letters, 10 digits, space, and simple punctuation marks (`,`, `.`, `!`, `?`, `'`). The set includes any keyslot required to type these characters on either QWERTY or Dvorak. The QWERTY training sentences lack the `;` keycode. However, Dvorak maps the letter `s` to the `;` keyslot. The model includes this keyslot to evaluate Dvorak. Conversely, the model excludes the QWERTY `-`, `[`, and `]` keyslots. Neither layout places a modeled character on these positions (Dvorak maps them to the unmodeled `[`, `/`, and `=`). The model does not treat Shift as a separate keystroke. It encodes Shift as a binary feature on the shifted keyslot, discussed in Section 3.3. Figure 1 shows the 42 modeled keyslots on a US QWERTY layout.

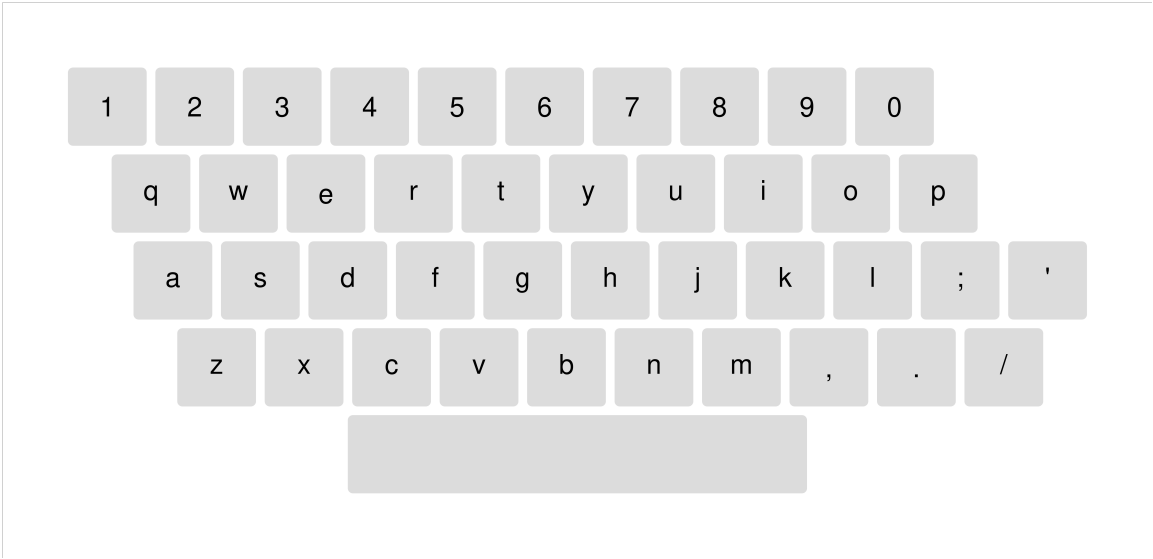

Figure 1: The 42 keyslots corresponding to the modeled keycodes, shown on a US QWERTY layout.

3.1.2 Corpus

The IKI model needs realistic text to estimate the character n-gram frequencies used as features. It also needs realistic text to evaluate typing systems such as abbreviation dictionaries on text people actually write. The corpus aims to represent what is typed on a keyboard.

It combines two sources in equal parts. Reddit comments from the Pushshift dataset [11] supply informal, modern writing. WikiText-103 [12], drawn from Wikipedia articles, supplies formal writing. Mixing the two spans a range of formality rather than a single register.

The pipeline keeps only Reddit comments from 2023 onward, to reflect modern language. To avoid any single community dominating, the pipeline samples Reddit sentences across randomly chosen subreddits, with at most 250 sentences taken from each.

A limitation is that Reddit text is often typed on a phone rather than a physical keyboard, which makes it less representative of keyboard typing. Including WikiText partly offsets this, since it is formal text likely written on a keyboard.

The pipeline applies cleaning: it strips URLs and characters outside a standard typing set, and a simple filter removes bot, moderator, and spam comments. The filter is imperfect, so some such text remains. The pipeline drops single-word fragments, since a sentence of one word carries almost no key-to-key transition signal.

The pipeline splits each text into sentences; each sentence becomes one corpus entry. Splitting by sentence approximates the larger chunks people type in, since a sentence is a rough unit of fluent typing. This is not fully realistic: typists sometimes stop mid-sentence, and sometimes type fluently straight across a full stop, so a sentence boundary does not always mark a real pause.

The pipeline filters sentences to 3–70 characters. It samples 50 000 sentences from each source, for 100 000 in total.

3.2 Data preparation

3.2.1 Participant filtering

The pipeline filters the data to fast typists (above 80 WPM) who use 9–10 fingers and are US-based.

The subset is motivated as follows:

- **Reduced variance.** Fast typists vary less in their IKI than slow typists [1], which gives the model a cleaner signal.
- **Matching the target users.** The model targets skilled, fast typists, so training on them matches the population it predicts for.
- **Simpler enrichment.** Assuming 9–10 fingers fixes which finger presses each key, simplifying the finger and hand features added during enrichment.
- **Consistent layout.** Restricting to US participants avoids layout variants such as UK-QWERTY or German QWERTZ, which place punctuation and symbols on different keys and would need more complex labeling. The data records country rather than physical layout, so US participants are assumed to use the US QWERTY layout.

The 80 WPM threshold trades typing speed against data volume. 80 WPM is assumed fast enough to represent the target population while retaining enough training data. Of the 56 469 US QWERTY participants who use 9–10 fingers, 8 853 (16%) type above 80 WPM, but only 1 449 (3%) type above 100 WPM. A 100 WPM cutoff would leave roughly six times fewer participants, risking too little data to train and validate the models reliably.

A matching subset of fast Dvorak typists serves as a held-out benchmark for cross-layout generalisation.

3.2.2 Typos

Real typing contains errors, which must be handled before the data can train a model. A modified Levenshtein distance identifies five typo types by comparing the raw input against the target sentence:

1. **Substitution** — a wrong key (e.g. “c i t” for “cat”).
2. **Insertion** — an extra key (e.g. “c i a t”).
3. **Deletion** — a skipped key (e.g. “c t”).
4. **Transposition** — two keys swapped (e.g. “c t a”).
5. **Proofreading** — any sequence using Backspace to correct a mistake.

When a typo is detected, the pipeline splits the sequence at the error. The split discards the IKI immediately before and after the typo, and the following correct keystrokes start a new segment. The model therefore trains only on sequences where the typist executed the intended motor plan. Of the 4 961 134 annotated QWERTY keystrokes, the filter removes 194 879 (3.93%) as typos.

Three reasons motivate this approach:

- **Simplicity.** Removing typos is simpler than modeling them, which would require the model to predict both an IKI and an error probability, alongside the time to recover from each typo type.

- **Sparse, noisy signal.** Typos are only 3.93% of keystrokes, and their recovery timing is erratic, so there is too little consistent data to learn from reliably.
- **Cleaner motor signal.** Discarding error-recovery intervals leaves only the motor flow the model is meant to predict.

The approach rests on two assumptions. The first is that typing speed and error rate are correlated, so optimizing for speed already avoids error-prone movements. The second is that typos are layout-independent. Neither is tested here. If either fails, removing typos could bias the model: it might rate a fast but error-prone key transition as good, having seen that transition only when typed correctly, or it might skew the cross-layout QWERTY-to-Dvorak comparison. Mistyped transitions also carry real motor movement, such as the "oz" sequence pressed when "foxes" is mistyped as "fozes", which the approach discards rather than learns from.

For example, a typist intending "foxes" but typing "fozes" results in the keystream shown in Table 3.

Key	f	o	z	e	s
Typo?	0	0	1	0	0
IKI (ms)	NaN	110	150	120	105

Table 3: A keystroke sequence containing a typo on the third key.

The pipeline splits this to remove the typo, leaving two clean segments shown in Table 4.

Key	f	o
IKI	NaN	110

Key	e	s
IKI	NaN	105

Table 4: The segments remaining after the typo and its surrounding intervals are removed.

3.2.3 Outlier detection and z-scores

Keystroke timings contain non-typing noise, such as pauses from external distraction. An IKI more than 4 standard deviations (SD) above a participant's mean counts as an outlier, and the sequence splits at that point, as with typos. This keeps 98.98% of IKI. A per-participant threshold, rather than a fixed cutoff, accounts for differing skill levels. No cutoff cleanly separates distraction from genuine typing. The 4-SD value is a subjective judgement rather than a data-derived one.

After outlier removal, each participant's IKI becomes a z-score:

$$z = \frac{x - \mu_p}{\sigma_p}$$

where μ_p and σ_p are the mean and standard deviation of that participant's non-outlier IKI. Per-participant normalization removes absolute speed differences between typists, so the model learns the relative difficulty of key transitions rather than individual typing speed. It also gives each participant unit variance, so a typist with a wider spread of IKI no longer carries disproportionate weight in the model's training loss.

After all filtering and cleaning, the data comprises 4 766 251 QWERTY keystrokes from 7 944 participants and 10 071 Dvorak keystrokes from 17 participants.

3.3 Data enrichment

Each keystroke is enriched with features computed from the typed text. The features split into two groups: motor features (key position, movement, finger and hand use) and linguistic features (frequency, word and syllable structure).

Most motor features assume standard touch typing: each key is always pressed by a fixed finger, with fingers resting on the home row `asdf hjk;`. Figure 2 shows the assumed finger assignment. Space is treated as `hand=2` (ambidextrous) since typists press it with either thumb.

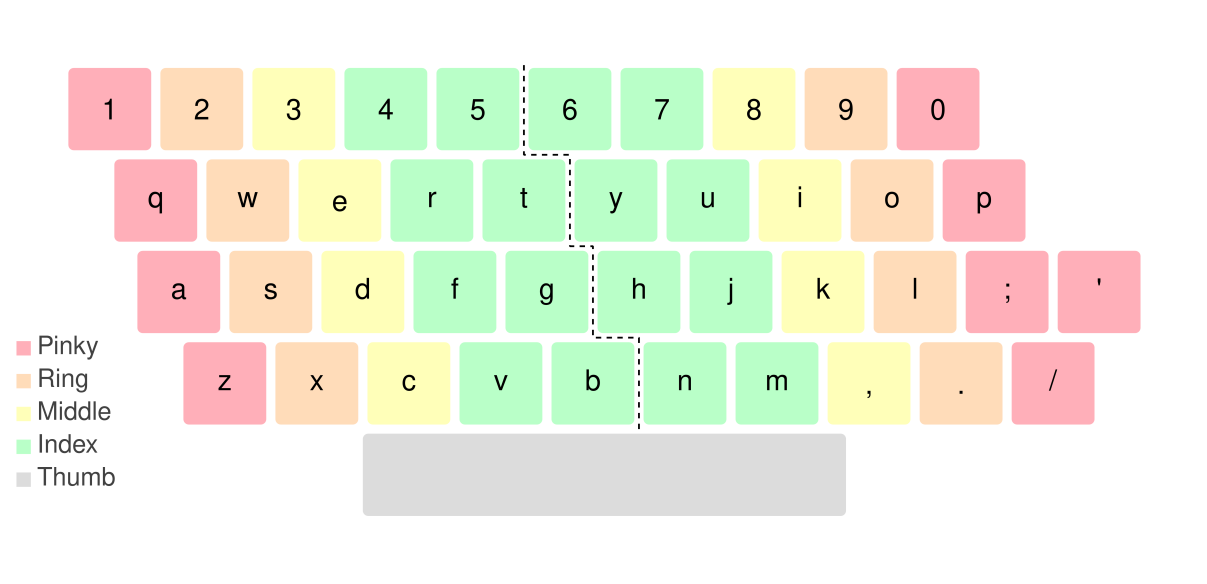

Figure 2: Assumed touch-typing finger map. Colors indicate finger type; the dashed line marks the left/right hand boundary.

Table 5 shows a sample of enriched features for one word.

Key	S	q	u	i	r	r	e	l
x	1.00	−0.25	3.25	2.25	2.75	2.75	1.75	1.00
y	0.00	1.00	1.00	1.00	1.00	1.00	1.00	0.00
Hand	0	0	1	1	0	0	0	1
Repetition	–	0	0	0	0	1	0	0
Out roll	–	1	0	1	0	0	1	0

Table 5: Selected enriched features for “Squirrel” on QWERTY.

3.3.1 Spatial position (x, y)

x and y encode a key’s position on the layout. Figure 3 shows the values assigned to each key on QWERTY.

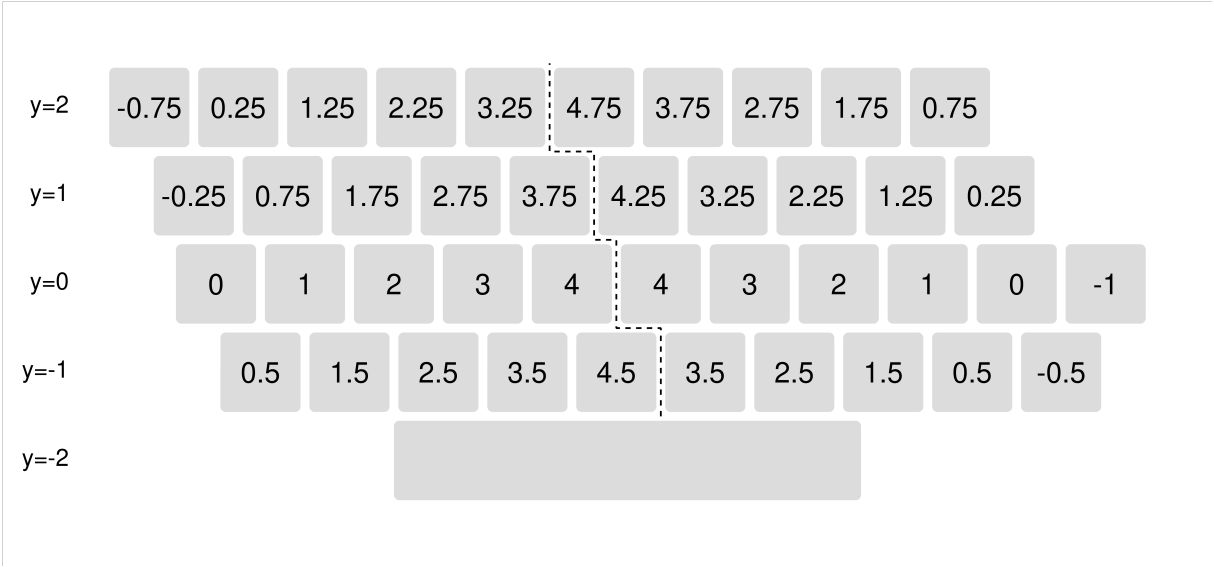

Figure 3: Key positions on QWERTY. `x` is symmetric across hands; `y=0` is the home row.

`y=0` is the home row, `y=1` the top row, `y=-1` the bottom row, and `y=2` the number row. Both `x` and `y` are encoded as continuous features. The space bar sits at `y=-2`; this could arguably be treated as a distinct category rather than simply two rows below home.

`x` uses a symmetric encoding, mirrored across the two hands: the pinky home key on each hand receives `x=0`, increasing toward the index finger and turning negative for keys beyond the pinky. “a” (left pinky) and “;” (right pinky) therefore share the coordinate `x=0`. Cross-layout generalisation is the main motivation for symmetric encoding: “;” does not appear in training data, so without it the model would see no examples of the right-pinky home row position. On Dvorak, “s” sits on the “;”-slot and receives `x=0`, so the model applies what it learned from “a” directly.

3.3.2 Movement (move_dist, move_sin, move_cos)

Three features encode how far and in what direction the finger moves to reach each key. Movement is a bigram feature: each key’s movement is measured relative to the previous key, so the first key of a sequence has no movement value.

The starting position depends on whether the same finger typed the previous key. For a same-finger bigram — one finger types two keys in a row, e.g. “rt” (both left index) — the finger is still at the previous key, so `move_dist` is the Euclidean distance from “r” to “t” in key-grid coordinates. For a different-finger bigram — e.g. “er” (left middle then left index) — the current finger starts from its home key: `move_dist` for “r” is the distance from “f” to “r”, not from “e” to “r”.

`move_sin` and `move_cos` are the sine and cosine of the movement angle. The angle is split into two components so that similar directions have close values: a raw angle would treat 1° and 359° as opposite extremes despite being almost the same movement.

3.3.3 Hand, finger, and finger type

`hand` is 0 for the left hand, 1 for the right hand, and 2 for space.

`finger` encodes which of the ten fingers presses a key (0 = left pinky, 1 = left ring, 4 = left thumb, 5 = right thumb, 9 = right pinky). `finger_type` collapses this to the finger category: pinky, ring, middle, index, or thumb. Both are derived from Figure 2.

All three are categorical, not ordinal. The linear and MLP models receive them one-hot encoded. LightGBM uses them as native categorical features.

3.3.4 Shift

`shift` flags a character that requires the Shift modifier (e.g. “A”, “!”). “A” and “a” share all position and finger features but differ on `shift`. A shift flag is used rather than a dedicated Shift-key column because the data does not record which hand pressed Shift.

3.3.5 Bigram and trigram indicators

These binary features flag motor patterns across two or three consecutive keys.

- **same_hand** — the bigram uses the same hand (e.g. “hi”).
- **same_finger** — the bigram uses the same finger (e.g. “nu”).
- **same_finger_trigram** — all three keys of the trigram use the same finger (e.g. “num”).
- **same_finger_skipgram** — the first and last keys share a finger and are different keys, while the middle key uses a different finger (e.g. “fur”: “f” and “r” are both left index; “u” is right index).
- **repetition** — the same key is pressed twice in a row (e.g. “rr”).
- **skipgram_repetition** — the same key appears at the first and last position of a trigram with a different key in between (e.g. “UwU”).
- **double_row_jump** — a same-hand bigram with one key on the bottom row (`y=-1`) and one on the top or number row (`y≥1`) (e.g. “cr”).

3.3.6 Rolls

A roll is a same-hand sequence of keys pressed by fingers moving in one consistent direction. An inward roll moves toward the index finger (pinky ⟶ index); an outward roll moves toward the pinky (index ⟶ pinky).

- **in_roll** — a same-hand bigram rolling inward (e.g. “at”, “ok”).
- **out_roll** — a same-hand bigram rolling outward (e.g. “ca”, “ts”).
- **in_triroll** — a same-hand trigram rolling inward continuously (e.g. “asd”).
- **out_triroll** — a same-hand trigram rolling outward continuously (e.g. “hil”).
- **redirects** — a same-hand trigram where the roll direction reverses (e.g. “cat”: “ca” rolls outward, “at” rolls inward).

3.3.7 Scissors

`scissors` flags a same-hand bigram where adjacent fingers — pinky, ring, or middle (not index or thumb) — reach across two or more rows (e.g. “ex”, “,o”).

3.3.8 Frequency features

Three features encode how often a character or sequence appears in text. Unigram and bigram frequencies are computed from a text corpus; word frequencies use the `wordfreq` library. Character and word frequencies follow a power law: a small number of items are very common and most are rare. Raw counts would give a highly skewed distribution; the Zipf scale, $\log_{10}\left(\frac{\text{count}}{\text{total}}\right) + 9$, compresses this into a bounded, more uniform range that is easier for a model to learn from.

- **unigram_frequency** — Zipf frequency of the single character.
- **bigram_frequency** — Zipf frequency of the two-character sequence, recorded at the second character.
- **word_frequency** — Zipf frequency of the whole word, with word boundaries at punctuation and whitespace.

3.3.9 Word and syllable position

These features encode where a character falls within its word and syllable. Word boundaries come from punctuation and whitespace; syllable boundaries are estimated by the `pyphen` library (e.g. “squirrel” ⟶ “squir-rel”).

- **word_index**, **word_length** — the character's position within the word and the word's total length.
- **word_relative_pos** — progress through the word, normalized from 0.0 to 1.0.
- **is_word_start**, **is_word_end** — binary flags marking the first and last character of a word.
- **is_syllable_start**, **is_syllable_end** — binary flags marking the first and last character of each estimated syllable.

3.3.10 Sequence position

These features encode where a character falls within the whole typed sequence.

- **sequence_pos** — the character's position in the sequence.
- **sequence_length** — the total number of characters in the sequence.
- **sequence_relative_pos** — progress through the sequence, normalized from 0.0 to 1.0.

3.4 Validation set and data splitting

Data splitting determines whether a model learns layout-independent motor costs or memorizes QWERTY timings. Because the training data is overwhelmingly QWERTY, a naive split causes the model to overfit to specific letter sequences. This split is an important design choice for cross-layout generalization.

3.4.1 Choosing the validation unit

The split must prevent data leakage at the word and bigram levels. Leakage occurs if the model sees a sequence during training and encounters it again during validation.

Word-level leakage versus sentences. Typing is chunk-based [10], and whole words approximate these chunks well. Sentence-level splitting is less effective because sentences share common words. Training on "The arctic fox" and validating on "The fennec fox" leaks the timings for the words "The" and "fox". Holding out whole words yields finer-grained validation units than sentences, and guarantees no validation word appeared in training.

Bigram-level leakage. Even with unseen words, models can leak information because bigrams are memorized across different words. If the model sees the bigram "th" in thousands of QWERTY instances of "the", it memorizes that specific motor rhythm. It can then apply this rhythm when predicting "th" in an unseen validation word like "that", bypassing biomechanical features.

Context leakage. Conversely, validating only at the bigram level introduces context leakage. Because typing is chunk-based, models can use the surrounding keystrokes of a word to predict the timing of a held-out bigram. Because of this context leakage, words serve as a better validation unit than bigrams. While word-level validation still allows some context leakage from the surrounding keys of unseen words, it leaks less data than validating only at the bigram level.

Alternative split approaches. Creating a validation set that holds out both specific words and specific bigrams simultaneously is infeasible because the validation sets would overlap heavily. Language is a densely connected graph, and this overlap makes strict cross-validation impossible. A diverse ensemble needs distinct, non-overlapping validation sets. 10 is a pragmatic choice. It gives enough models for a useful ensemble while leaving each fold ample training data. Training one model per fold gives 10 models, and the spread of their predictions estimates how uncertain the model is about a given input.

Alternatively, a sentence-level split combined with both word and bigram regularization might have worked better. However, it would have added complexity to the data pipeline, so it was avoided.

3.4.2 The splitting algorithm

The split uses two components: word-based validation folds and bigram-based training dropout.

Word-level validation folds. A greedy bin-packing algorithm divides the dataset into 10 non-overlapping validation folds. The algorithm sorts 50 000 unique words by descending total frequency. Processing them in that order, it assigns each word to the fold with the lowest running frequency total. This process distributes both highly common and rare words evenly across all folds, making each fold representative of the overall frequency distribution. The algorithm balances the 10 folds to within 0.001% of each other in total dataset rows. Any words outside the top 50 000 are assigned to a fold randomly.

Bigram-level dropout. The same greedy algorithm assigns 1 865 bigrams to 10 buckets based on descending frequency. During training on a given fold, every row whose bigram belongs to that fold's bucket is withheld. Each bucket is one tenth of all bigram occurrences. The model therefore never sees 10% of them during training. Combining word-level folds with bigram dropout leaves 81% (0.9×0.9) of the data for training, enough to learn the biomechanical patterns.

3.5 Models

Five models are trained on the same data, differing in architecture and feature set.

Linear Regression is an interpretable baseline. It sets a reference point: if the other models cannot beat it, their added cost is not justified. The Typability Index [4] uses the same linear approach, so it also anchors this work against prior art.

LightGBM is a feature-based model. A gradient-boosted tree ensemble over the engineered features.

MLP Main is the main neural network model, trained on engineered features.

MLP Linguistic trains on the linguistic features only. It aims to estimate how much of the accuracy comes from linguistic structure rather than learned motor patterns.

MLP DL takes the deep-learning approach: it strips away most feature engineering and learns from near-raw key positions, so the model must find motor patterns itself rather than being handed them.

All neural models use regularization, specifically dropout and weight decay. Regularization prevents the networks from simply memorizing the QWERTY training data, forcing them to distinguish generalizable motor patterns from linguistic patterns.

3.5.1 Loss function

The models minimise mean absolute error (MAE). MAE suits a keyboard-optimization goal because it weights every millisecond equally, unlike MSE, which over-penalises large errors and would let a few slow outliers dominate the objective.

3.5.2 Temporal window

To capture context, models do not just look at the current keystroke. They use a temporal window spanning W_{back} preceding keystrokes and W_{ahead} future keystrokes. If $W_{\text{back}} = 2$ and $W_{\text{ahead}} = 1$, the model predicts the time to type the current key at t using the features of keys at $t-2$, $t-1$, t, and $t+1$. This sliding window allows the models to account for the momentum of previous movements and the anticipatory cost of upcoming characters. The optimal window size is treated as a hyperparameter (Section 3.7).

3.5.3 Exploratory analysis

An initial evaluation tests multiple architectures on 10% of the QWERTY data. The evaluated models include 1D CNN, LSTM, Transformer, Random Forest, XGBoost, Linear Regression, LightGBM, and

MLP. LightGBM and the MLP architectures achieve the lowest error rates and train faster. Linear Regression performs well given its simplicity. It remains as a baseline. The remaining architectures are discarded. This exploratory phase also establishes the starting hyperparameters described below.

3.6 Feature selection

Feature selection simplifies the models, cutting inference time while keeping or even improving generalization. All selection runs on fold 0 only: per-fold selection would need an arbitrary rule to reconcile disagreements between folds, for negligible gain on a fold of roughly 10^6 samples.

The method depends on the architecture. Linear regression uses L1 regularization to drive uninformative coefficients to zero directly. LightGBM and the MLPs use SHAP-driven ablation against a `random_noise` control feature.

3.6.1 Interpreting SHAP

SHAP (SHapley Additive exPlanations) assigns each feature a signed, additive contribution to every prediction. The mean absolute SHAP value ranks how much the model relies on a feature: it measures the magnitude of influence, not its direction. Rankings are model-specific, so a feature LightGBM leans on may rank low for an MLP. A `random_noise` feature (standard normal, no signal) sets an objective cutoff: any real feature ranking below it carries no useful information. Correlated features split their SHAP credit, so a low value can mean a feature is redundant rather than uninformative. The reverse also holds: a high-SHAP feature can still be removable if other features carry the same signal. The ablation therefore tests each removal against validation MAE rather than trusting the ranking alone.

3.6.2 Linear regression

The linear baseline uses L1 (Lasso) regularization rather than SHAP-driven ablation. L1 is intrinsic to the model: the loss that fits the coefficients also drives uninformative ones to exactly zero, handling redundancy and the heavy multicollinearity of the one-hot finger/hand encoding at once. SHAP ablation ranks features one at a time and would keep both members of a correlated pair.

An `SGDRegressor` (`epsilon_insensitive` loss, $\varepsilon = 0$, equivalent to minimising MAE) trains on all candidate features with `penalty="l1"` and a fixed window ($W_{\text{back}} = 2$, $W_{\text{ahead}} = 1$). α sweeps from 10^{-4} to 10^{-1} on a log scale (10 values); the α minimising validation MAE defines the surviving features (non-zero coefficient in at least one temporal slot).

L1 dropped four one-hot finger columns (`finger_2/4/5/7`), `finger_type_4`, and two of the three one-hot hand columns, keeping most motor and linguistic features. This left 41 surviving features (listed in Appendix A.4).

3.6.3 LGBM

Feature selection uses SHAP rather than LightGBM's built-in split-gain importance. Split-gain importance is biased toward features used in early splits. SHAP, computed with `shap.TreeExplainer` on 50 000 validation samples, averages over all possible feature orderings, providing a stable measure of each feature's contribution regardless of split order.

A `LGBMRegressor` trains on all 36 candidate features (plus the `random_noise` control) with `objective='regression_l1'`, `n_estimators=500`, `w_back=2`, `w_ahead=1`, default hyperparameters otherwise, and `finger`/`hand` as categorical features. Selection runs on fold 0, with SHAP importances summed across all temporal slots ($t-2$ to $t+1$). The baseline reached MAE 0.5852 [95% CI: 0.5833, 0.5872] (full ranking in Appendix A.2.1).

Six features ranked below `random_noise` (`is_pad`, `same_finger_trigram`, `scissors`, `out_triroll`, `is_word_start`, `in_triroll`) and were removed unconditionally.

Beyond those, is_word_end is a binary flag already captured more richly by word_relative_pos; finger_type correlates with finger while ranking lower; and sequence_length with sequence_relative_pos both approximate information already in sequence_pos. Removing all four gave 26 features at MAE 0.5866 [95% CI: 0.5846, 0.5885], not meaningfully worse.

unigram_frequency correlates with both shift and bigram_frequency, suggesting redundancy. It also acts as a near-unique identifier for each key on QWERTY, which risks the model memorising per-key biases rather than learning generalisable motor difficulty. Removing it improved performance: 25 features, MAE 0.5743 [95% CI: 0.5723, 0.5762].

move_cos and move_sin correlate with x and y (0.72 and 0.97). For a linear model this multicollinearity would hurt, but LGBM handles it without penalty. Removing both, or either alone, consistently worsened results, so all four stay.

hand ranked weakest among the rest, and finger carries similar information, but removing hand gave MAE 0.5791 [95% CI: 0.5771, 0.5810], worse, so it stays. is_syllable_start and is_syllable_end correlate at 0.82 with word start/end, a distinction word_relative_pos already encodes continuously; removing them improved to 23 features at MAE 0.5746 [95% CI: 0.5727, 0.5766]. word_index relates to word_relative_pos and word_length, but removing it gave MAE 0.5807 [95% CI: 0.5788, 0.5827], worse, so it stays.

SHAP was rerun on the 23-feature set (MAE 0.5771 [95% CI: 0.5751, 0.5790]; ranking in Appendix A.2.2). redirects now ranked lowest, but removing it gave MAE 0.5791 [95% CI: 0.5771, 0.5811], worse. The 23 features are final (listed in Appendix A.4).

3.6.4 MLP Main

Because an MLP learns differently from a tree, it gets its own SHAP analysis. A multilayer perceptron (512 → 256 → 64 → 1, GELU activations, 0.2 dropout) trains on all 51 candidate features (plus random_noise), with finger, hand, and finger_type pre-expanded to one-hot columns. Training uses the AdamW optimizer, MAE loss, and a learning rate that halves when validation loss plateaus. The inputs are standard-normalized so that features with different scales contribute equally to the gradient updates. It trains with a batch size of 32 768, up to 15 epochs with early stopping at patience 2. The window is w_back=3, w_ahead=1, on fold 0. SHAP values, computed for the neural network using 300 background and 700 test samples, are summed across all 5 temporal slots. The baseline reached MAE 0.5758 [95% CI: 0.5737, 0.5778] (full ranking in Appendix A.2.3).

Rather than ablating from scratch, the LGBM feature set is the starting point, since both models learn from the same data and their important features overlap. Expanding finger and hand to one-hot columns gave 34 features at MAE 0.5737 [95% CI: 0.5717, 0.5757].

The MLP rated is_syllable_start and is_syllable_end higher than LGBM did. This is expected: a tree can threshold word_relative_pos to recover word boundaries implicitly, whereas an MLP benefits from the explicit binary signal. Adding both improved to MAE 0.5724 [95% CI: 0.5704, 0.5744]. With the syllable features now encoding position, word_index became redundant; removing it improved to 35 features at MAE 0.5714 [95% CI: 0.5695, 0.5734].

Adding finger_type alongside the finger one-hot columns, on the reasoning that it encodes finger-category symmetry explicitly, gave 40 features at MAE 0.5734 [95% CI: 0.5714, 0.5754], worse, so it was not added.

Unlike LGBM, MLPs are sensitive to highly correlated inputs, since redundant features can confuse gradient updates. move_sin correlates 0.97 with y and move_cos correlates 0.72 with x. Removing move_cos and y, keeping the more symmetric move_sin and x, gave MAE 0.5729 [95% CI: 0.5710,

0.5749], slightly worse. Adding `move_cos` back alone gave 0.5758, and `y` back alone gave 0.5733, both worse than 0.5714, so all four stay.

`sequence_pos` ranked low, so `sequence_relative_pos` was tried as a replacement; the result of 0.5724 [95% CI: 0.5704, 0.5744] was not better, so `sequence_pos` stays.

SHAP was rerun on the 35-feature set (ranking in Appendix A.2.4). `sequence_pos` now sat just above `random_noise`, but removing it gave MAE 0.5756 [95% CI: 0.5736, 0.5776], worse. `finger_7` ranked near the bottom, but removing it gave 0.5757 [95% CI: 0.5737, 0.5777], worse. `word_relative_pos` overlaps with the syllable features, but removing it gave 0.5726 [95% CI: 0.5706, 0.5746], worse. The 35-feature set is final at MAE 0.5714 [95% CI: 0.5695, 0.5734] (listed in Appendix A.4).

3.6.5 MLP Linguistic

The model keeps the 7 mostly-linguistic features from the MLP Main set (listed in Appendix A.4).

An MLP is used rather than LGBM for this variant: exploratory analysis showed LGBM did worse on this linguistic-only subset, so an MLP is better suited to this small, mostly continuous feature set.

This is only an approximation of a pure linguistic baseline. Many features that would normally count as linguistic are excluded because they correlate too strongly with motor features and would leak motor information back in. For example, `unigram_frequency` near-uniquely identifies each key on a fixed layout: knowing `unigram_frequency=0.093` is enough to infer the key is "e", at which point the model can learn the motor cost of pressing "e". The retained features are the linguistic signals with the lowest motor leakage; no clean separation exists, so more linguistic signal is available in principle than this variant can capture without confounding.

3.6.6 MLP Deep Learning

The feature set keeps mostly raw inputs (listed in Appendix A.4).

The motor inputs are not fully raw. `hand` is a human-derived heuristic, but it is necessary: the layout encoding is deliberately symmetric, so `x` and `y` do not uniquely identify a key, and two keys (one per hand) can share the same `(x, y)`. Without `hand` the model cannot tell which finger produced an event, which cripples its ability to learn motor patterns. The symmetric coordinate encoding is kept rather than switched to absolute positions, because the mirror structure should aid generalisation across layouts that share it. The variant is therefore not "fully deep learning". Linguistic features are identical to MLP Linguistic.

3.7 Hyperparameter optimization

All models use Optuna for hyperparameter optimization. Optuna uses a Tree-structured Parzen Estimator (TPE) sampler, a form of Bayesian optimization: it fits a probabilistic model over past trial results and uses it to propose the next hyperparameters most likely to improve the objective, rather than sampling randomly or searching a grid. The four non-linear models (LGBM, MLP Main, MLP DL, MLP Linguistic) follow a two-phase strategy: phase 1 runs a small trial budget for each candidate $(W_{\text{back}}, W_{\text{ahead}})$ window in a grid; phase 2 continues the winning window's study to refine the same hyperparameters. Both phases share a search space and a `MedianPruner`, which stops a trial whose intermediate score falls below the running median, freeing compute for promising configurations.

Window size is the dominant structural decision, since it sets the input dimensionality. Fixing the best window first lets phase 2 explore one consistent search space instead of spreading the budget across inferior windows.

LinReg uses a single phase: its search space has only five mostly categorical hyperparameters, training is fast, and there are no architectural decisions, so a flat grid over windows with 20 trials

per config characterizes it. The full search spaces and final hyperparameter configurations for all models are listed in Appendix A.3.

Each window-analysis table reports the first N trials per window in chronological order (N = phase-1 budget), excluding the phase-2 trials appended to the winning window. `num_completed` counts finished trials; `num_pruned` counts trials the `MedianPruner` stopped early (LinReg has no pruner). Where the best-MAE window was not chosen, the reasoning follows the table.

3.7.1 Linear Regression

The linear baseline uses `SGDRegressor` with `epsilon_insensitive` loss ($\varepsilon = 0$, equivalent to minimising MAE), `StandardScaler` normalization, and one-hot encoded categorical inputs (`finger`, `hand`, `finger_type`). The feature set is `LINREG_FEATURES` from the L1-selection step (Appendix A.4). Training uses `max_iter=1000` and `tol=1e-3` with no early stopping.

W_{back}	W_{ahead}	best MAE	avg MAE	completed
3	1	0.5924	0.6122	20
2	1	0.5930	0.6267	20
1	1	0.5934	0.6042	20
3	0	0.5940	0.6282	20
2	0	0.5946	0.6121	20
1	0	0.5948	0.6300	20

Table 6: Phase 1 window analysis for Linear Regression (20 trials per window).

Table 6 shows that 3/1 had the best MAE, but 1/1 was chosen: its best MAE is within 0.001 of 3/1 (below typical run-to-run noise), it has the lowest avg MAE of any config (0.604, a more robust optimum), and it is the smallest input.

3.7.2 LGBM

Gradient-boosted trees on `LGBM_FEATURES`, with `finger` and `hand` passed as categorical features.

Fixed training settings: `objective='regression_l1'`, `metric='l1'`, up to `n_estimators=1000` with early stopping at `patience=50` rounds on the validation set.

W_{back}	W_{ahead}	best MAE	avg MAE	completed	pruned
3	1	0.5624	0.5636	6	5
3	2	0.5629	0.5727	9	3
2	2	0.5633	0.5647	5	7
2	1	0.5637	0.5641	5	7
3	0	0.5659	0.5676	5	7
1	1	0.5661	0.5732	5	6
1	2	0.5661	0.5700	8	4
2	0	0.5680	0.5743	6	6
1	0	0.5685	0.5727	8	4

Table 7: Phase 1 window analysis for LGBM (12 trials per window).

As shown in Table 7, 3/1 had the lowest best MAE and was selected for phase 2. The next contenders (3/2 and 2/2) were within 0.001 but added input dimensionality without clear benefit.

3.7.3 MLP Main

The main neural model uses a multilayer perceptron with variable depth and width. Training uses the AdamW optimizer, MAE loss, and a learning rate that halves when validation loss plateaus (patience of 1 epoch). Inputs are standard-normalized to aid training stability, and categorical inputs are one-hot encoded.

Fixed training settings: up to 30 epochs, early stopping patience 3 epochs.

The window grid was narrowed to $\{2, 3\} \times \{1, 2\}$ based on LGBM phase 1 and exploratory analysis, which ruled out $W_{\text{back}} = 1$ and $W_{\text{ahead}} = 0$ as inferior across all configs, saving five window configs of trials.

W_{back}	W_{ahead}	best MAE	avg MAE	completed	pruned
2	1	0.5595	0.5637	8	4
3	1	0.5599	0.5634	6	6
2	2	0.5615	0.5661	9	3
3	2	0.5629	0.5660	7	5

Table 8: Phase 1 window analysis for MLP Main (12 trials per window).

Table 8 indicates that 2/1 had the best MAE and is the smallest input, making the choice straightforward.

Resulting architecture: `Linear(140, 1024) -> GELU -> Dropout(0.08) -> Linear(1024, 1024) -> GELU -> Dropout(0.08) -> Linear(1024, 1)`. Input dimension is 35 features $\times$ 4 timesteps $= 140$. Total trainable parameters: ≈1.20M.

3.7.4 MLP DL

MLP DL uses the same architecture and pipeline as MLP Main, but on the smaller `MLP_DL_FEATURES` set. The depth range extends to 1–5 layers so the network has more capacity to learn motor patterns from raw coordinates. The same narrowed window grid applies.

Fixed training settings: same as MLP Main.

W_{back}	W_{ahead}	best MAE	avg MAE	completed	pruned
2	2	0.5630	0.5727	9	3
2	1	0.5633	0.5752	9	3
3	1	0.5634	0.5694	7	5
3	2	0.5641	0.5780	8	3

Table 9: Phase 1 window analysis for MLP DL (12 trials per window).

Table 9 shows that 2/2 had the best phase-1 MAE, but 2/1 was chosen: the difference (0.0003) is small, and 2/1 is significantly simpler.

Resulting architecture: `Linear(44, 256) -> GELU -> Dropout(0.10) -> Linear(256, 256) -> GELU -> Dropout(0.10) -> Linear(256, 256) -> GELU -> Dropout(0.10) -> Linear(256, 1)`. Input dimension is $11 \text{ features} \times 4 \text{ timesteps} = 44$. Total trainable parameters: ≈143k.

3.7.5 MLP Linguistic

MLP Linguistic uses the same architecture and pipeline as MLP Main, but on `MLP_LINGUISTIC_FEATURES` only. Because the feature set is much smaller, the per-window trial budget drops to 8. The same narrowed window grid applies.

Fixed training settings: same as MLP Main.

W_{back}	W_{ahead}	best MAE	avg MAE	completed	pruned
2	1	0.6323	0.6390	6	2
3	2	0.6333	0.6375	8	0
2	2	0.6341	0.6428	6	2
3	1	0.6359	0.6385	5	3

Table 10: Phase 1 window analysis for MLP Linguistic (8 trials per window).

Table 10 reports that 2/1 had the best MAE and is the smallest input.

Resulting architecture: `Linear(28, 1024) -> SiLU -> Dropout(0.39) -> Linear(1024, 1024) -> SiLU -> Dropout(0.39) -> Linear(1024, 1)`. Input dimension is $7 \text{ features} \times 4 \text{ timesteps} = 28$. Total trainable parameters: ≈1.08M.

3.8 Final training

Every model trains on the full dataset with 10-fold cross-validation, using the feature set and hyperparameters from the preceding sections. This produces an ensemble of 10 submodels per architecture, whose predictions average at inference time.

The spread across folds supplies an estimate of epistemic uncertainty. The cost is proportionally higher inference time and storage.

For the linear model and the MLP variants, a separate standardisation (z-score normalization) is fitted on each fold's training data and stored with its submodel. This prevents data leakage: the validation fold's statistics never inform the scaler. LGBM needs no normalization, so it stores no scaler.

3.9 Evaluation

Each model is an ensemble of 10 submodels, evaluated on QWERTY and zero-shot on Dvorak.

Validation MAE scores every QWERTY keystroke with the one submodel that never trained on it, then pools these out-of-fold predictions and measures the error once. This is a single-submodel score. Every QWERTY keystroke trained 9 of the 10 submodels, so only one submodel is leakage-free per keystroke. The full ensemble therefore cannot be scored on QWERTY without a held-out set (Section 5.3).

Dvorak is fully held out, so all 10 submodels are leakage-free on it. This gives two numbers that differ only in the order of two steps: averaging the 10 submodels, and measuring the error. The single-model Dvorak MAE measures each submodel's error, then averages the 10 errors. It is computed the same way as Validation MAE, so the two are directly comparable and measure cross-layout transfer. The ensemble Dvorak MAE reverses the order: it averages the 10 submodels into one prediction, then measures that prediction's error. Averaging predictions cancels some error, so the ensemble usually scores lower, and it is the predictor the applications use.

A second metric, bias, is the mean signed error: the average of predicted minus measured IKI. Unlike MAE, it keeps the sign, so it captures whether the model over- or under-predicts on average rather than how far off it is. Bias is the same for the single-model and ensemble predictors, because bias is a linear function of the predictions: the average of the errors equals the error of the average. MAE uses absolute error, which is nonlinear and breaks that equality.

The baseline always predicts zero, the mean of z-score-normalized IKI by construction, so its error marks the performance of a model with no predictive power.

3.9.1 Confidence intervals

A bootstrap quantifies the uncertainty in the reported statistics. The procedure resamples the data with replacement 2 000 times and takes the 2.5th and 97.5th percentiles of the resampled statistic as a 95% confidence interval.

The resampling unit is the sentence, not the individual keystroke. Keystrokes within a sentence are correlated: the sliding feature window overlaps between neighbours, and typing rhythm links successive intervals. Resampling individual keystrokes treats these correlated values as independent, which underestimates the uncertainty and narrows the interval. Resampling whole sentences preserves the within-sentence correlation, which widens the intervals.

Sentence resampling can also overcorrect. Since the feature window spans only a few keystrokes, keys far apart in a long sentence are nearly independent; whole-sentence clustering discards these degrees of freedom, widening the interval more than correlation alone requires.

A stricter design would also resample participants. Each typist has their own patterns, so keystrokes from the same person are correlated. Resampling sentences leaves this correlation in the data. This narrows the interval.

4 Results

4.1 Model performance

Table 11 reports QWERTY validation MAE and two zero-shot Dvorak MAEs for each model: the single-model average (with 95% CI) and the deployed ensemble (Section 3.9).

Model	Val MAE	Val R^2	Dvorak MAE (single, 95% CI)	Dvorak MAE (ensemble, 95% CI)	Dvorak R^2
Baseline	0.7374	0.0000	0.7411	0.7411	0.0000
Linear Regression	0.6014	0.2329	0.6125 (0.5972, 0.6269)	0.6106 (0.5961, 0.6252)	0.2401
LightGBM	0.5678	0.3041	**0.5922** (0.5780, 0.6053)	**0.5884** (0.5752, 0.6027)	**0.2938**
MLP (Main)	**0.5657**	**0.3051**	0.5998 (0.5860, 0.6138)	0.5958 (0.5819, 0.6098)	0.2828
MLP (DL)	0.5718	0.2920	0.6038 (0.5897, 0.6184)	0.5987 (0.5848, 0.6139)	0.2711
MLP (Ling)	0.6324	0.1881	0.6231 (0.6075, 0.6380)	0.6153 (0.6007, 0.6304)	0.2371

Table 11: Model performance on QWERTY validation and zero-shot Dvorak transfer. The single Dvorak MAE is the mean of the 10 submodels' individual MAEs; the ensemble Dvorak MAE scores the averaged prediction (Section 3.9). MAE is normalized to participant-wise IKI standard deviation.

All models achieve lower MAE than the baseline. MLP Main achieves the best QWERTY validation MAE, with LightGBM marginally behind. LightGBM has the smallest gap between its validation and single-model Dvorak MAE (0.024 vs. 0.034 for MLP Main). The ensemble lowers Dvorak MAE by about 0.004 over the single-model average, a small gain because the submodels nearly agree (fold std ≈0.002). MLP DL achieves similar MAE to the main models despite fewer engineered features, suggesting the raw coordinate representation captures meaningful motor signal.

4.2 Bias analysis

Table 12 reports the bias and bias shift for each model. Bias is the model's mean signed error, the average of predicted minus measured IKI. A negative bias means the model predicts faster typing than measured. The bias shift is the Dvorak bias minus the validation bias: it measures how much this systematic error changes between layouts, after removing any misprediction common to both. A bias shift of zero means the error is identical on both layouts.

Model	Val bias	Dvorak bias	Bias shift (95% CI)
Linear Regression	−0.1234	−0.1044	+0.0190 (+0.0136, +0.0248)
LightGBM	−0.0769	−0.0754	**+0.0015** (-0.0039, +0.0068)
MLP (Main)	**−0.0631**	**−0.0505**	+0.0126 (+0.0071, +0.0179)
MLP (DL)	−0.0902	−0.0823	+0.0079 (+0.0025, +0.0135)

Table 12: Mean signed error (bias) and its shift across layouts. Units are normalized Z-scores.

All models have negative biases on both QWERTY and Dvorak, meaning they systematically predict lower IKI than measured (faster typing than reality) for both populations. MAE optimization on a right-skewed IKI distribution causes this; it is unrelated to layout.

The bias shift captures the layout-specific effect after removing this baseline tendency. All shifts are positive, meaning the under-prediction is smaller on Dvorak than on QWERTY: the models predict Dvorak as slightly slower than ground truth warrants. The models are biased against Dvorak in layout comparisons.

LightGBM is the only model whose 95% CI includes zero, so its bias shift is not statistically significant. Its shift is also the smallest, less than 0.08 ms at the dataset's IKI standard deviation of 50.4 ms.

4.3 Ensemble uncertainty

The 10-fold ensemble provides a natural measure of epistemic uncertainty: the standard deviation of the 10 submodel predictions per keystroke. A well-calibrated ensemble disagrees more on unfamiliar inputs and less on familiar ones. Values are computed over 25 000 sentences (1 009 622 keystrokes), sampled from a combined Wikipedia/Reddit corpus.

Model	QWERTY std	Dvorak std	Colemak std	Random std
Linear Regression	0.4352	0.4444	0.4468	0.4304
LightGBM	0.0941	0.0896	0.0892	0.1162
MLP (Main)	0.1210	0.1245	0.1227	0.1287
MLP (DL)	0.1078	0.1048	0.1046	0.1270

Table 13: Mean ensemble standard deviation (epistemic uncertainty) across layouts.

Table 13 reports this statistic across layouts. The diagnostic question is whether each model's disagreement scales with input familiarity, increasing on a random layout and staying low on structured layouts. Comparing Random std to QWERTY std:

- **LightGBM** (0.1162 vs. 0.0941, +23%) and **MLP DL** (0.1270 vs. 0.1078, +18%) show separation.
- **MLP Main** (0.1287 vs. 0.1210, +6%) shows less separation.
- **Linear Regression** (0.4304 vs. 0.4352, −1%) is uninformative as an uncertainty estimator.

4.4 Inference speed

Table 14 reports timing on 1 009 622 keystrokes (25 000 sentences). Linguistic feature precomputation (16.7 s) is a shared one-time cost not included in the per-model totals.

Model	Layout enrich (s)	Predict (s)	Total (s)
Linear Regression	0.777	9.032	9.809
LightGBM	0.637	105.682	106.319
MLP (Main)	0.677	15.330	16.007
MLP (DL)	**0.543**	**4.710**	**5.253**

Table 14: Inference time for 1 million keystrokes on Google Colab (T4 GPU for MLPs, default CPU for Linear Regression and LightGBM).

MLP DL requires 5.3 s, benefiting from a small feature set (≈143k parameters vs. ≈1.2M for MLP Main) and efficient GPU batching. Linear Regression requires 9.8 s despite running on CPU, since the model itself is cheap. MLP Main is slower due to a larger feature matrix and wider network. LightGBM requires 106.3 s: tree traversal does not parallelize efficiently on GPU, and the large ensemble (382 812 leaves) drives most of this cost.

5 Discussion

5.1 Interpreting the results

5.1.1 Zero-shot evaluation works

Prior data-driven typing models trained and evaluated on QWERTY alone, leaving cross-layout transfer untested. This model is the first evaluated zero-shot on a different physical layout. The transfer holds: the single-model MAE rises by at most 0.034 moving from QWERTY validation to Dvorak.

5.1.2 LightGBM is the most layout-neutral model

Bias shift measures how much a model's systematic error changes between QWERTY and Dvorak (Section 4.2). LightGBM's bias shift, +0.0015, is the only one whose confidence interval includes zero, so it is statistically indistinguishable from zero (under 0.08 ms). The other three models carry a significant shift, though each stays under 1 ms. All shifts are positive, so the models under-credit Dvorak. LightGBM therefore transfers across layouts with the least bias.

5.1.3 The models learn motor signal

The near-linguistic model (MLP Linguistic) sets a reference for how much accuracy comes from language statistics alone. The motor models beat it on validation and on Dvorak. This indicates they learned motor patterns beyond language, and these survive the move to Dvorak.

5.1.4 Motor signal is recoverable from raw positions

MLP DL does almost as well as the other motor models from key positions alone, without hand-designed motor features. The motor signal is therefore partly recoverable from near-raw spatial input.

5.1.5 Typing is nonlinear

The nonlinear models beat Linear Regression on both layouts. Two effects could explain this. Features may interact: the cost of one movement may depend on another. Features may also scale nonlinearly: a 2 cm finger movement may not cost twice a 1 cm movement.

5.1.6 MLP Main overfits

MLP Main fits the QWERTY training data best but transfers worst. Its capacity is the likely cause. Many features and a large network let it memorize QWERTY-specific patterns instead of a general motor rule. Those patterns fail once the letters move. MLP DL is simpler on both counts, fewer features and a smaller network, so it has less room to memorize. It generalizes better.

5.1.7 Typing is anticipatory

Window size selection tests how far ahead each model looks (Section 3.7). For Linear Regression and LightGBM, looking one key ahead beat looking only backward. Typing is therefore anticipatory. The MLPs were not tested without lookahead during HPO. They showed the same effect during the exploratory phase, so they are presumed to behave the same. This selection ran on QWERTY only, but the effect likely holds on Dvorak too.

5.1.8 A performance ceiling remains

No model exceeds 0.31 R^2. One explanation is aleatoric noise. Typing timing varies even for the same keystroke in the same context. Some of each IKI may be unpredictable in principle. How much of the residual is irreducible noise rather than unmodeled signal is unclear, and this project does not measure it.

5.2 Use cases

The models predict per-keystroke IKI for any sequence of modeled keys on QWERTY, and on Dvorak with a measured bias. A use case is trustworthy when its keystrokes stay within this domain. Four applications meet this condition.

1. **Dvorak versus QWERTY speed.** Both layouts have real typing data, and the Dvorak bias is measured (Section 4.2).
2. **Abbreviation dictionaries.** An abbreviation is typed as a QWERTY sequence. Predicting its per-keystroke cost is similar to the task the model was validated on.
3. **One-shot shift.** It encodes motor patterns the model has been trained on.
4. **Repeat key.** So does the repeat key.

Ensemble disagreement also offers a signal for when to distrust a prediction. On a random layout, LightGBM and MLP DL raise their disagreement by 23% and 18% over QWERTY (Section 4.3). Both respond to out-of-distribution input. MLP Main raises its disagreement by only 6%, so it carries a weaker signal.

The signal is partial, not clean. For the same two models, Dvorak std is slightly lower than QWERTY std, even though Dvorak is also zero-shot (Table 13). One interpretation is that Dvorak's letter placement involves fewer high-variance motor patterns than QWERTY.

5.3 Limitations

5.3.1 Systems it cannot evaluate

Chording presses several keys at once to produce a single output. The model predicts the interval between sequential single-key presses. A chord has no interval within it. Its surrounding transitions never appear in the training data. Neither is predictable. The limit is the data, not the method. The same approach, retrained on recordings of chorded typing, could presumably predict typing speed on chorded keyboards.

A layer key works like Shift: held down, it remaps the other keys to a different set. For example, holding space could turn the home-row keys `asdf` into `1234`. The model represents only one such modifier, Shift. The training data contains no other layer key, so the model cannot learn the cost of holding an arbitrary one. Evaluating layers would require data that records them.

5.3.2 Untested domains

The training data covers standard English prose on standard row-staggered keyboards. It lacks keystrokes for function keys (F1–F12), rare symbols, and modifiers other than Shift. Typing patterns may also differ across languages. Furthermore, the model's performance on alternative physical keyboards, such as ortholinear or split designs, is untested. Generalization to these domains remains unknown.

5.3.3 Other layouts

The model runs on any layout, but its bias is known only for Dvorak. On Colemak, an optimized layout, or a random one, the bias is unmeasured. If it matched the small Dvorak bias, the model could rank these layouts. How reasonable that assumption is stays unknown. A ranking could therefore be correct without any way to confirm it, and wrong without any way to detect it. Measuring bias on more layouts would settle this, but that needs typing data on each layout.

5.3.4 Data

After filtering to fast, US, 9–10-finger typists, the training data is almost entirely QWERTY: 7 944 participants against 17 on Dvorak, or 4 766 251 keystrokes against 10 071. The Dvorak sample is

small, which widens the confidence intervals; a larger sample would resolve the bias shifts more precisely. Dvorak typists are also self-selected: people who chose to learn a non-default layout may differ from the QWERTY population in typing habits or motivation, introducing a confound the data cannot control for.

The data is real typing, so the linguistic confound is not fully removed. Common sequences may be fast because they are practiced, not because they are physically easier. Explicit linguistic features mitigate this but cannot eliminate it.

The model targets skilled typists. Transfer to slower typists is untested. Slower typists are noisier [1]. Many of the features assume standard touch-typing, so training on them would likely be harder.

5.3.5 No held-out QWERTY set

A QWERTY dataset should have been held out from every fold, so the ensemble could be scored on QWERTY as it is on Dvorak. None was, so the QWERTY number is a single-submodel out-of-fold score (Section 3.9) and underestimates the deployed ensemble.

5.3.6 Validation context leakage

Word-level folds still expose the surrounding keys of an unseen validation word. The model can use that context to predict the held-out timings. The QWERTY validation MAE is therefore likely slightly optimistic. A sentence-level split with word- and bigram-level dropout would reduce this leakage. The Dvorak evaluation uses fully held-out data and is unaffected.

5.3.7 Typos

Typos make up 3.93% of keystrokes and are slow, so discarding them rather than modeling them is a substantive simplification. The approach assumes that speed and error rate correlate, and that typos are layout-independent. Neither is tested, so whether discarding them biases the results is unknown.

5.3.8 LightGBM inference speed

LightGBM takes 106.3 s per million keystrokes, an order of magnitude slower than the other models (Table 14). Layout optimization requires scoring many candidate layouts against a corpus. The inference speed makes LightGBM impractical for that optimization.

5.4 Future work

The main open problem is training a layout-unbiased model. Only LightGBM shows no statistically significant cross-layout bias; the other three models carry a small but significant shift. A speed difference between layouts may be smaller than a model's bias: a 0.5% Dvorak speedup is undetectable if the model carries 1% layout bias. Two directions could narrow that gap.

5.4.1 Better data

Two approaches could provide better data.

Collecting real typing data on several layouts would almost certainly cut bias. Even one more layout would help considerably. Three layouts let one serve for training, one for validation, and one for test. This also simplifies data splitting and feature selection

A cleaner route removes the confounds at the source. Participants type randomly generated sequences until speed plateaus; the plateau time estimates motor cost. Typing nonsense removes the linguistic familiarity confound. Training to plateau removes the layout familiarity confound. Unlike isolated-digraph tapping [9], random sequences preserve the influence of surrounding keys. A model trained on such data estimates motor cost directly, free of the linguistic and practice biases this project can only partly control. The approach requires a dedicated data-collection study rather than

reuse of existing keystroke logs. How much data such a study needs is unclear. Learning curves estimated from existing data could bound it.

5.4.2 Better architecture

The fixed-window input is overly complex. Letting some window positions carry fewer features would be simpler, faster, and less prone to overfitting. This was not done because it is harder to implement. MLP DL, with its simpler feature set, showed less overfitting than MLP Main, suggesting simpler architectures generalize better.

5.4.3 Confidence intervals

The intervals resample sentences, capturing correlation within a sentence. They do not resample participants. Correlation between keystrokes from the same typist is therefore not modeled. Accounting for it is future work.

6 Model Applications

Typing systems affect typing speed. Faster typing increases productivity. Analyzing system performance guides choices that improve productivity.

This section contains four independent studies. Each study applies the trained model to answer a specific question.

All studies assume proficiency at the typing system. None of the studies account for the cost of learning a new system.

6.1 One-Shot Shift

6.1.1 Introduction

Typing capital letters is slower than typing lowercase letters. Standard capitalization requires pressing the Shift key and the target letter simultaneously. One-shot shift (1SS) modifies this: tapping a dedicated one-shot key shifts only the next keystroke, then the modifier releases. Pressing the one-shot key then `c` outputs `C`. The mechanism was originally developed to assist typists with motor impairments [13, p. 147]. It eliminates the simultaneous press but adds an additional keystroke. The net effect on typing speed is untested.

6.1.2 Methodology

This study evaluates 1SS using the trained LightGBM model. 1SS is scored on QWERTY, but the one-shot key is a novel input. Its motor transition is familiar, a home-row key followed by a letter, but its specific feature encoding is not. The prediction is therefore a mild extrapolation, which makes robustness the deciding criterion. LightGBM transfers across layouts with the least bias (Section 4.2), the best available evidence of robustness, so it is selected.

The model scores a corpus sample twice: once with standard capitalization and once with isolated capitals rewritten using 1SS. An isolated capital is a single capital letter with no adjacent capital. This includes word-initial capitals like the “F” in “Fox” and mid-word capitals like the “H” in “GitHub”, but excludes all-caps runs like “NASA”. The evaluation uses 25 000 sentences from the Reddit and Wikipedia corpus (Section 3.1.2), containing 41 154 isolated capitals. The difference in total predicted time between the 1SS and standard versions represents the cost or saving of 1SS.

Evaluating 1SS requires assigning the one-shot key to a physical slot. Standard Shift keys lack motor features in the model because the 136M Keystrokes dataset [1] does not record which Shift key was used. The semicolon slot serves as the one-shot key. It is absent from the evaluation corpus and occupies a home-row position with a low predicted movement cost.

The semicolon never appears in the training data. Its motor features mirror the a key on the opposite home row, so the model can still predict its movement cost. The linguistic features are the harder part. This study uses an In-Word encoding: the model treats the one-shot key as the first character of the word. The one-shot key receives the word-start features, and the capitalized letter becomes the second character. Table 15 compares the features for "Fox" under both systems.

(a) Standard

Key	F	o	x
Shift	1	0	0
Bigram freq	6.06	5.37	5.08
Word freq	4.66	4.66	4.66
Word start	1	0	0
Rel. pos	0.00	0.50	1.00

(b) One-shot shift (In-Word Encoding)

Key	1SS	f	o	x
Shift	0	0	0	0
Bigram freq	7.58	6.06	6.45	5.08
Word freq	4.66	4.66	4.66	4.66
Word start	1	0	0	0
Rel. pos	0.00	0.33	0.67	1.00

Table 15: Enriched features for "Fox" typed normally (a) and with one-shot shift (b). Each word is preceded by a space, so the first key's bigram frequency is its word-initial (space → key) value. The In-Word encoding treats the one-shot key as the start of the word, shifting the relative positions of subsequent letters.

This encoding is chosen because it keeps the post-space slot occupied by a word-start. In-Word makes the one-shot key that word-start, so the slot still carries a word frequency. The Boundary alternative places the one-shot key between the space and the word, where it belongs to no word and has no word frequency. The model has never seen a key without a word frequency in the post-space slot, making that transition out-of-distribution (OOD). In-Word is therefore less OOD. It also simplifies the encoding of a word with a mid-word capital, like the "H" in "GitHub".

A capital at the start of a sentence has no preceding interval in the dataset. Scoring that interval would unfairly favor the standard version, so the 1SS version drops it too. Sentence-initial capitals are therefore scored as free under both schemes. The measured saving comes only from isolated capitals that are not sentence-initial.

6.1.3 Results

Over 25 000 sentences, 1SS is 0.71% faster than standard capitalization. Each non-initial isolated capital saves an average of 18.5 ms. Extending the same saving to sentence-initial capitals would raise the overall speedup to 1.1%.

6.1.4 Discussion

The interpretation is that 1SS yields a speedup on the semicolon slot. Whether it survives placing 1SS on the Shift key itself is unclear. The semicolon may be a faster slot, though the two Shift keys allow hand alternation.

The prediction depends on the In-Word feature encoding. A different encoding of the one-shot key could change the result. It is unclear which encoding best models how a typist mentally chunks the key. Without behavioral data, this cannot be confirmed.

LightGBM was chosen as the most robust model, but MLP Main and MLP DL could have been scored on 1SS too. Their predictions would test whether the saving is robust across architectures or specific to LightGBM.

The 0.71% figure is conservative. Sentence-initial capitals are scored as zero saving because the dataset segments sentences, removing the first keystroke's preceding interval. In continuous prose such a capital is preceded by a space, like any word-initial capital, so 1SS should save there too. The 1.1% estimate assumes it saves the 18.5 ms, which this evaluation cannot confirm.

6.1.5 Conclusion

Assuming the typist mentally chunks the modifier as part of the word and the one-shot key occupies a fast position, one-shot shift is faster than standard capitalization. On QWERTY, 1SS reduces typing time by an average of 18.5 ms per isolated capital, yielding a 0.71% overall speedup, or about 1.1% if the benefit extends to sentence-initial capitals.

6.2 Repeat Key

6.2.1 Introduction

A repeat key is a dedicated key that re-emits the previously typed character. It converts same-finger repeated keypresses, such as typing "poppy" as `p o p RPT y` or "squirrel" as `s q u i r RPT e l`, into alternating-finger sequences. Standard typing requires rapidly lifting and pressing the same finger twice. The repeat key removes this motion. The effect on overall typing speed depends on how frequently double letters occur and how fast the alternating sequence is.

6.2.2 Methodology

This study evaluates repeat key speed using the trained LightGBM model. LightGBM transfers to unfamiliar inputs with the least layout bias and the best Dvorak MAE (Section 4.2). The evaluation uses the 25 000-sentence Reddit and Wikipedia corpus (Section 3.1.2).

Evaluating the repeat key requires assigning it to a physical slot. This study assumes the repeat key (`RPT`) occupies the right-pinky home slot, displacing the semicolon `;`. The semicolon does not appear in the evaluated sentences, so replacing it with a repeat key leaves them fully typable. The result depends on this slot choice; the repeat key's value on other slots is untested.

The repeat key introduces sequences that do not exist in standard typing. Because the `RPT` key lacks real-world bigram data, the evaluation computes its bigram frequencies from the occurrences of literal double letters in the corpus. For example, the bigram `t` followed by `RPT` takes the same frequency as `tt`. The word frequency features remain unchanged. The model scores the corpus twice: once with standard double letters and once with double letters rewritten to use the repeat key. The difference in predicted time represents the speedup of the mechanical change.

6.2.3 Results

Over the 25 000 sentences, using the repeat key for all double letters is 0.26% faster than standard typing.

6.2.4 Discussion

The 0.26% rests on a fixed assumption: the repeat key takes the `;` slot. How it would perform on other slots is untested.

The repeat key needs little extrapolation: its bigram frequencies come from real double letters and the word frequencies are unchanged. MLP Main scored best on QWERTY, so it may be more accurate here too and might have scored the repeat key differently than LightGBM. Scoring the repeat key under the other models would test whether the 0.26% holds across architectures and is left for future work.

Another value of the repeat key is that it frees the physical double-tap gesture. Because the repeat key handles literal double letters, a physical double-press (e.g., tapping `r` twice) is no longer needed for ordinary spelling. This gesture becomes available for other functions, such as keyboard shortcuts, macros, or outputting a full word.

When these freed double-taps are used for word output, a dictionary of 23 such entries reaches a 1.52% speedup (Section 6.3).

6.2.5 Conclusion

On the right-pinky home slot, the repeat key increases typing speed by 0.26%. It also unlocks the physical double-tap gesture for secondary functions, such as outputting full words.

6.3 Abbreviation Dictionary Optimization

6.3.1 Introduction

Abbreviated typing replaces short strings with the longer text they stand for: whole words or common word-endings. The typist enters an abbreviation, and an expansion engine writes out the full form. The hard part is choosing the abbreviations. A good one is fast to type and easy to remember.

Speed is a difficult criterion. The typing time of an arbitrary key sequence cannot be measured without collecting new data for every candidate. The trained IKI models supply it instead. They predict the typing cost of any sequence, so thousands of candidates can be compared.

Such expansion should raise typing speed, but by how much is unknown. This section quantifies it.

6.3.2 Methodology

Every abbreviation needs an expansion method: a way to type it and trigger the replacement. This choice comes first, because it sets which abbreviation strings are possible at all. The rest of the pipeline then has five stages: vocabulary extraction, candidate generation, intuitiveness scoring, speed precomputation, and optimization. The first three build a pool of scored candidates. The last two select a dictionary from it.

6.3.2.1 Expansion Methods

Two extra keys are added to a standard QWERTY keyboard. Each key gives one way to expand an abbreviation.

The first method uses a **trigger key** (`TRG`). An abbreviation fires when `TRG` follows its string. So `"th" + TRG` expands to `"the "`, with a trailing space. Plain `"th"` inside a word is left untouched. The trigger key removes any ambiguity with normal typing, so every abbreviation can use it. `TRG` is a thumb key beside the spacebar. It displaces no letter, and the model gives it the spacebar's position and finger.

The second method reuses the **repeat key** (`RPT`) of Section 6.2. Because `RPT` already types literal double letters, the plain double-press of a key is freed for other uses. The expansion engine claims it: a double-press fires an expansion directly, with no trigger key. This is the `doubletap` form. A double-press of `t`, for example, can expand to `"the "`.

The repeat key changes only how a *doublechar* abbreviation, which is a doubled letter such as `"tt"`, is triggered. Its available trigger forms depend on the repeat regime:

1. **Repeat key off.** One form exists: the plain `"tt" + TRG`.
2. **Repeat key on.** The plain form is dropped for two alternatives. The `doubletap` form is a double-press of `t`. The `rpt_trg` form is `t` then `RPT` then `TRG`.

With the repeat key on, the two forms can map to two different words. Claiming double-presses has a cost. A literal double letter inside a word, such as the `"rr"` in `"squirrel"`, must come from `r` then `RPT`, or it triggers an accidental expansion. `RPT` is assumed to sit at the right-pinky home slot, displacing `;`.

Both placements are fixed assumptions, not optimized. The trigger and repeat keys are modeled as real keystrokes with their own typing cost, not as free actions.

6.3.2.2 Vocabulary

The vocabulary holds two entry types, drawn from the combined Reddit and Wikipedia corpus of 100 000 texts (Section 3.1.2).

Words are the 3 000 most frequent tokens, matched with the regex `[a-z]+(?:'[a-z]+)*` (ASCII lowercase; contractions kept whole).

Suffixes are word endings of 2 to 5 characters. A suffix qualifies if it ends at least 50 distinct words. This keeps real word parts (`-tion`, `-ing`, `-ment`) and drops noise endings. The count of distinct words uses Wikipedia text only. Reddit posts run words together (`"activelooking"`), which would inflate the count for false endings. Ranking by total frequency still uses the full corpus. The 500 most frequent suffixes are kept. Overlapping ones such as `-ation` and `-tion` both stay, and the optimizer chooses between them.

A word or a suffix is called an *entry*. The pipeline treats the two alike: each entry is abbreviated, scored, and selected the same way.

6.3.2.3 Candidate Generation

Each entry generates abbreviation candidates by seven strategies:

1. **Deletions** — every single-character deletion, applied recursively to `min_len = 1`, capped at 5 000 results per entry.
2. **Replacements** — one or two letters of the entry swapped for random ones. The result is kept only when it is at most three characters long. This admits short combinations like `"tx"` for `"text"` and blocks long random strings.
3. **Phonetic deletion-then-replace** — the entry is first shortened by deletion, then some remaining letters are swapped for phonetically similar ones, producing candidates like `"kat"` for `"category"`. Deletion alone (strategy 1) cannot change letters, and replacement alone (strategy 2) does not shorten past three characters, so this combination reaches forms that neither produces.
4. **Random deletion-then-replace** — the same, with random letters in place of phonetic ones.
5. **Syllable** — the entry is split into syllables; both the syllable acronym (`"without"` → `"wo"`) and the first full syllable (`"without"` → `"with"`) are emitted.
6. **Doublechar** — for each character in the entry, the doubled string is emitted (`"the"` → `{"tt", "hh", "ee"}`).
7. **Letter-name** — a word that sounds like a letter name yields that letter (`"you"` → `"u"`).

A candidate must be at least 25% shorter than its entry (`len(abbr) ≤ 0.75 × len(entry)`); doublechar candidates are exempt. Up to 8 000 candidates per entry survive deduplication.

6.3.2.4 Intuitiveness Scoring

Intuitiveness is scored by a hand-built heuristic. It combines 11 weighted components.

Weight	Component	Description
0.25	contiguity	Whether the whole abbreviation is an in-order subsequence of the entry, and how few letters it skips. A gapless block scores 1, each skipped letter multiplies the score by 0.9, and a non-subsequence scores 0. The higher of the raw-letter and phonetic-normalized score is taken.
0.15	first_char	Whether `abbr[0] == entry[0]`.
0.10	is_prefix	Whether the abbreviation is a prefix of the entry.
0.10	prefix_utility	Quadratic score peaking when the abbreviation is ~40% the length of the entry. It is short enough to save keystrokes, yet long enough to be recognizable.
0.10	literal_seq_sim	Each abbreviation letter scored by its closest entry letter, then averaged; position is ignored. `"sqrl"` for `"squirrel"` scores 1, since `s`, `q`, `r`, `l` all occur.
0.10	phonetic_seq_sim	Same as above after phonetic normalization.
0.10	syl_acronym	Whether the abbreviation matches the syllable acronym, which is the first letter of each syllable (e.g. `"without"` ⟶ `"wo"`, `"squirrel"` ⟶ `"sr"`).
0.08	first_letter_bonus	Bonus for single-letter abbreviations, scaled down for longer entries.
0.08	compression	Reward proportional to characters saved, capped at 8. Short sequences are more memorable; this is unrelated to speed.
0.03	edit_dist_sim	$1 -$ normalized Levenshtein distance over the whole strings, so position matters. `"sqrl"` for `"squirrel"` scores 0.5.
0.03	last_char	Whether `abbr[-1] == entry[-1]`.

Table 16: Intuitiveness score components and their weights.

Phonetically related characters count as partly similar, not different: `k/c/q` are near-identical, `s/z/c` sound alike, and `f/v` and `p/b` are voiced pairs. This lets `"kat"` score better against `"category"` than `"iat"`. A penalty then counts abbreviation characters that pair with no entry character, even a phonetically similar one. One unmatched character multiplies the score by 0.2, and two or more by a floor of 0.1. So `"kat"` for `"category"` keeps its score, while `"clzscficati"` for `"classification"` collapses near zero. The result is clipped to $[0, 1]$.

Word	Abbreviation	Score
otter	ot	0.93
rhododendron	rho	0.87
fox	fx	0.73
squirrel	sl	0.64
mango	ngo	0.51
poppy	pi	0.44
panda	pth	0.39
fennec	vec	0.24
fluffy	fyy	0.10
meow	cat	0.01

Table 17: Example intuitiveness scores.

6.3.2.5 Speed Precomputation

For every entry and every candidate, the DL and LightGBM ensembles predict the typing cost: the sum of per-keystroke IKI Z-scores over the keystream. The saving is the difference.

$$\text{savings} = \text{entry\_cost} - \text{abbr\_cost}$$

Each saving is measured in real context, not in isolation. Up to six occurrences of the entry are sampled from the corpus. Because they are drawn from real text, common surroundings appear in proportion to how often they occur. Each occurrence keeps the entry's real neighbors around it, as far back and ahead as the model can see. The entry keystream and the abbreviation keystream share those neighbors, so the saving compares them in the same surroundings. Scoring every candidate at every position in the corpus would be too slow, so six samples are the compromise. The candidate's saving is the mean over its samples. These samples serve only the optimizer; the final evaluation instead re-scores each abbreviation inside whole corpus sentences.

Each sample also carries an uncertainty. The 10-fold ensemble returns a standard deviation alongside its mean, and these are pooled into one uncertainty per candidate. A noisy candidate is later penalized for it.

The model reads two frequency features: the bigram frequency and the word frequency. An abbreviation has neither. Its `TRG` and `RPT` keys have never been typed, so no real bigram involves them. And the abbreviation string itself has no corpus frequency.

The trigger-key frequency is circular. How often a `TRG` bigram occurs depends on which abbreviations the dictionary holds. That dictionary is the optimizer's output, so the frequency cannot be known before the optimizer runs.

Optimization therefore uses placeholders. Every trigger bigram takes the mean of all bigram frequencies. The approach assumes it biases every abbreviation pattern about equally, so their relative savings stay roughly correct.

Evaluation uses real values instead. Once the dictionary is fixed, each trigger bigram's true frequency is computed from it: the bigram is as frequent as the entries that fire through it. The final speedup uses these real, precomputed frequencies, not the placeholder. A literal double letter, such as the `"tt"` in `"letter"`, is typed as `t` then `RPT`; that bigram takes the frequency of `tt`. The values are computed once, not iterated, since the dictionary is not re-optimized against them.

The word frequency works the same in both stages. A word entry takes the frequency of the word it abbreviates. A suffix entry takes the frequency of the word it appears in. This treats the abbreviation as just as familiar as that word.

6.3.2.6 Model Choice and Goodhart Avoidance

Two models score speed: the deep MLP ensemble (DL) and LightGBM. Both predict per-keystroke IKI far better than a keystroke-count baseline ($R^2 \approx 0.29$ versus 0).

The dictionary is optimized with DL and evaluated with LightGBM. Using one model for both inflates the result: the optimizer exploits that model's errors, and the same model then rewards them (Goodhart's law). A second model as the judge reduces this. LightGBM is not fully independent, since it trains on the same IKI data as DL. But it is a different kind of model, using gradient-boosted trees rather than a neural network, so it makes different errors. The optimizer cannot exploit errors the judge does not share. The baseline dictionary, the configuration used throughout, scores 13.29% under DL, its own model, and 12.99% under LightGBM, the judge. The 0.30-point gap is the inflation an in-sample score would report. The LightGBM score is the one reported throughout.

6.3.2.7 Optimization

Dictionary selection is a binary Integer Linear Program (ILP) solved with Google OR-Tools CP-SAT. A variable $x_{w,a} \in \{0, 1\}$ marks whether entry w takes abbreviation a. The objective trades speed against intuitiveness:

$$\max \sum_{w,a} f_w x_{w,a} \left[(1-\lambda)\left(s_{w,a} - k_\sigma \sigma_{w,a}\right) + \lambda \cdot 10 \cdot \text{intuit}_{w,a} \right] - \sum_{(b,a)} f_b s_a z_{b,a}$$

f_w is corpus frequency, $s_{w,a}$ the mean predicted Z-score saving, $\sigma_{w,a}$ the ensemble standard deviation, and $\lambda \in [0, 1]$ the trade-off weight. At $\lambda = 0$ the objective is pure speed. At $\lambda = 1$ it is pure intuitiveness. The uncertainty penalty k_σ (default 1.0) subtracts one standard deviation, so a noisy prediction ranks below a confident one of equal mean. A candidate is admitted only if two things hold: its intuitiveness clears a floor of 0.3, and its uncertainty-penalized saving $s_{w,a} - k_\sigma \sigma_{w,a}$ stays positive.

Three hard constraints apply:

1. At most one abbreviation per entry.
2. No two entries share the same `(abbreviation, trigger form)` pair.
3. The dictionary holds at most N entries.

The final term prevents double counting when two selected suffixes overlap, such as `"ion"` and `"on"`. A word ending in `"ion"` — like `"dandelion"` — is matched by the longer suffix first, so `"ion"` claims it. If both suffixes are selected, `"on"`'s saving would also be credited on those words, counting the gain twice. The term subtracts the shorter suffix's own saving s_a once, weighted by the longer suffix's frequency f_b. The indicator $z_{b,a}$ is 1 only when both are selected. Here s_a is the same quantity as $s_{w,a}$, but for the shorter suffix's entry.

The penalty is only an estimate, and it leans toward over-punishing. It values the shorter suffix at its best candidate rather than the one actually chosen, so it subtracts a little more than the true overlap. Valuing each suffix at a single representative candidate also keeps the optimization small enough to solve exactly.

The true objective is harder than this. It would score each abbreviation in full sentence context, and the dictionary would itself change the trigger-key frequency. That problem cannot be solved exactly. The approach simplifies it instead. It uses sampled-context savings, fixed candidate scores, and synthetic trigger frequencies, which makes the objective linear. CP-SAT then solves this simplified

problem to its exact optimum. That optimum is the best dictionary for the simplified problem, not necessarily for the true one. Evaluation re-runs the model in full context to measure the real speedup.

6.3.2.8 Top-K Candidates per Word

Each word offers its K best abbreviations to the optimizer. The shortlist is ranked by the same speed–intuitiveness blend the objective uses at the current λ. With $K = 1$ each word is locked to its single best abbreviation. Two words can then collide: `"t"` may be the top pick for both `"the"` and `"that"`, yet only one can use it. With $K > 1$ the optimizer has fallbacks. It can keep `"t"` for `"the"` and send `"that"` to its next-best form. A larger K never hurts the optimizer's own objective, since more options can only help it. But it multiplies the candidate count, so solver time and memory bound how large it can be.

K	Speedup
3	7.25%
5	9.13%
8	11.50%
10	12.23%
12	12.99%
15	12.75%

Table 18: Speedup versus candidates per word K ($\lambda = 0.1$, $N = 160$, LightGBM evaluation).

Table 18 shows speedup rising steeply with K, peaking at $K = 12$ with 12.99%. $K = 15$ scores slightly lower, 12.75%, even with strictly more candidates. The extra candidates only help the optimizer's own objective, scored with DL. They do not necessarily help the LightGBM evaluation. They also give the optimizer more room to exploit DL's errors, the same Goodhart effect the two-model setup guards against. That exploitation does not transfer to the LightGBM judge. So past $K = 12$ the dictionary can look better to the optimizer yet score worse in evaluation. $K = 12$ is the compromise used throughout: it gives the peak measured speedup, and a larger K only adds solver cost.

6.3.2.9 Evaluation

The dictionary is applied to 25 000 sentences, typed with and without it. Each sentence becomes one keystream, so the model sees the full surrounding context for each keystroke. Each token is matched by greedy longest-match: whole-word entries beat suffixes, and the longest suffix wins.

Matching is case-insensitive, and capitalization carries to the abbreviation's first letter. With `ca → cat`, the token `"Cat"` is typed as `"Ca"` then the trigger. The shift cost appears on both sides of the comparison, so capitalization does not bias the saving.

Punctuation attached to the start or end of a token is separated before lookup, so the bare word is matched and the punctuation typed around the abbreviation. Apostrophes count as word characters, so `"don't"` stays whole. When punctuation follows an expansion, *smart-space deletion* drops the trailing space the trigger would otherwise insert: `"squirrel."` is typed as the abbreviation, its trigger, then the period, and the removed space costs no keystroke.

For a matched token, the word-frequency feature is set to the parent word's Zipf value, not the abbreviation's. The model should see how familiar the real word is. Keystreams are scored in batches.

6.3.3 Results

The optimizer is run across λ from 0 to 1 at fixed $K = 12$ and $N = 160$. Each run yields a dictionary, evaluated on the same sentences. Figure 4 plots the frontier: mean intuitiveness against measured speedup, one point per λ.

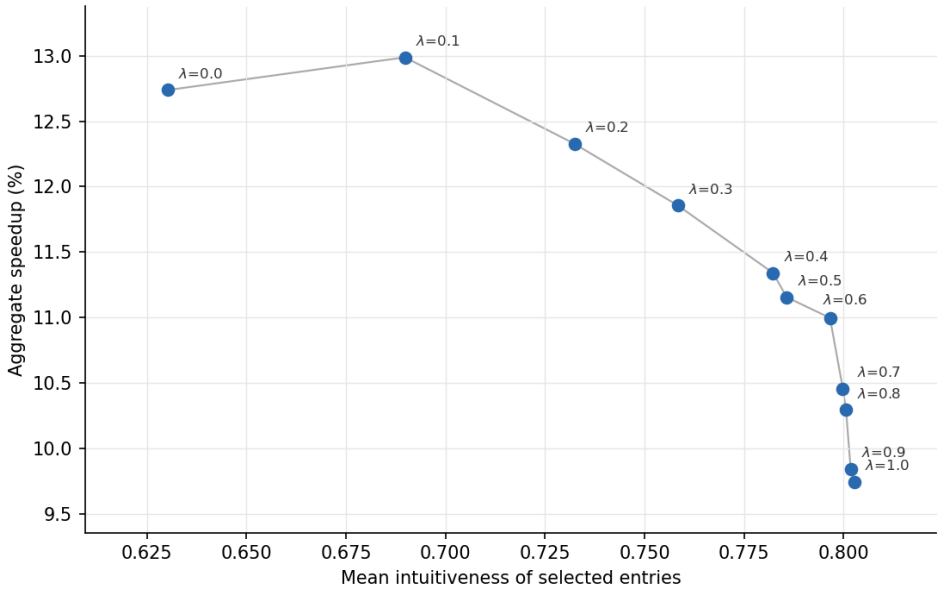

Figure 4: Pareto frontier of the λ sweep. Each point is a dictionary optimized at a different λ ($K = 12$, $N = 160$), evaluated with LightGBM. $\lambda = 0$ is pure speed, $\lambda = 1$ pure intuitiveness.

The frontier is a trade-off. Speedup peaks at $\lambda = 0.1$, 12.99%, and falls steadily to 9.74% at $\lambda = 1$, a span of 3.25 percentage points. Over the same range mean intuitiveness rises from 0.63 to 0.80. $\lambda = 0.1$ is the baseline used throughout: it gives the highest measured speedup while already raising intuitiveness from 0.63 ($\lambda = 0$) to 0.69. Past this point, each further gain in intuitiveness costs measurable speed, because the per-word shortlist itself is λ-aware and begins offering more intuitive but slower candidates.

Dictionary size sets a second trade-off. Figure 5 sweeps N at fixed $\lambda = 0.1$ and $K = 12$. More entries cover more tokens and raise the aggregate speedup.

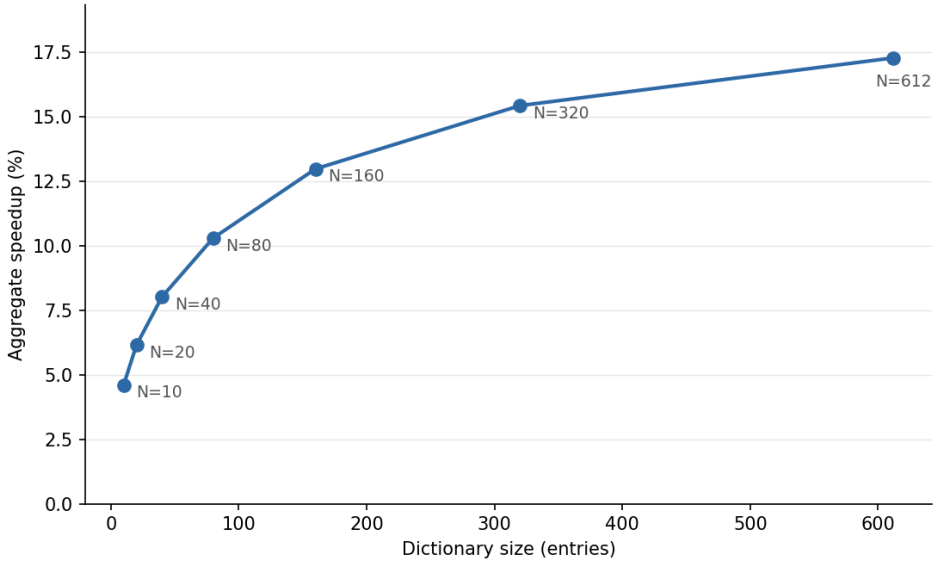

Figure 5: Aggregate speedup against dictionary size N at $\lambda = 0.1$, $K = 12$.

N	Speedup
10	4.61%
20	6.15%
40	8.02%
80	10.30%
160	12.99%
320	15.44%
612	17.29%

Table 19: Speedup versus dictionary size N ($\lambda = 0.1$, $K = 12$). The last row is the largest dictionary reachable, not a requested size.

Speedup rises steeply, then flattens. The first 160 entries reach 13.0%. Adding more raises it to 17.3%, but the dictionary saturates at 612 entries (Table 19). At $\lambda = 0.1$ only that many words have an abbreviation that clears every filter, so a request for 640 or 1 280 both return 612. Beyond this point the limit is the candidate pool, not N.

Table 20 shows some of the highest-impact entries. Frequent short words take single-letter abbreviations, and common endings such as `-tion` and `-ing` shorten across many words at once.

Entry	Abbreviation
the	t
was	w
you	u
and	d
-ation	an
-ted	e
-ing	in

Table 20: Example entries from the $\lambda = 0.1$, $N = 160$ dictionary, among the highest-impact abbreviations. A leading - marks a suffix.

6.3.3.1 Contribution of the Repeat Key

The two expansion methods can be separated by re-optimizing with some trigger forms switched off (Table 21). Every variant uses the same baseline ($\lambda = 0.1$, $K = 12$, $N = 160$, LightGBM evaluation).

Configuration	Speedup
Full dictionary (TRG + repeat-key forms)	12.99%
TRG forms only (repeat key unused)	12.79%
Double-tap only (no TRG key)	1.52%
Empty dictionary (repeat regime, no expansions)	0.26%

Table 21: Speedup with the trigger forms restricted, isolating the repeat key's contribution.

The full dictionary reaches 12.99%. Dropping the two repeat-key forms lowers this to 12.79%, a difference of 0.20 percentage points; the remaining entries are all plain `abbr + TRG` strings. A dictionary built from double-taps alone reaches 1.52%, as only 23 words have a usable doublechar abbreviation. An empty dictionary, which still types literal double letters through RPT, scores 0.26%. Most of the speedup comes from the trigger key, not the repeat key.

The TRG-only variant still runs on repeat-key hardware, with double letters typed through RPT. A keyboard with no repeat key at all, where ; was restored and double letters were typed plainly, was built and scored. It reaches 12.53%, against 12.79% for TRG-only and 12.99% for the full dictionary.

6.3.3.2 Contribution of Suffixes

Suffixes are isolated the same way, by re-optimizing with whole words or suffixes dropped (Table 22). The baseline is again $\lambda = 0.1$, $K = 12$, $N = 160$, LightGBM evaluation.

Configuration	Speedup
Full dictionary (words + suffixes)	12.99%
Whole words only	12.33%
Suffixes only	3.18%

Table 22: Speedup with whole-word or suffix entries restricted, isolating the suffix contribution.

Suffixes add 0.66 percentage points over a words-only dictionary. Most of the speedup comes from whole words.

6.3.4 Discussion

The method works as a demonstration. The IKI model picks abbreviations that measurably speed typing, and the optimizer turns thousands of scored candidates into a usable dictionary. This shows the model can drive abbreviation design.

Speed and intuitiveness trade off measurably, a 3.25-point span across the frontier. The candidate pool softens the conflict: candidates are generated to look like their words, not at random, so the strategies tend to produce intuitive abbreviations.

The size curve points to a different limit. The limit on a useful dictionary is not the model, but how many abbreviations a person will learn. The candidate pool also caps out at 612 entries at this floor: beyond that, more candidates are needed, for example by lowering the intuitiveness floor.

The optimizer is crude. But it limits the dictionary's quality, not the trustworthiness of the speedup number. Its shortcuts are several:

- **Vocabulary cap.** Only the 3 000 most frequent words and 500 suffixes are eligible, so a gain from a rarer entry is never captured.
- **Candidate pool.** Each entry keeps at most 8 000 candidates from seven strategies, a small slice of the possible abbreviations, so a better abbreviation that no strategy proposes is never scored.
- **Shortlist.** Each word offers only its $K = 12$ best candidates to the optimizer. That cutoff is tuned at one setting, not proven optimal, and its best value may shift with dictionary size and λ.
- **Sampled context.** Savings come from a few sampled contexts, not full text.
- **Fixed inputs.** Candidate scores are fixed and trigger frequencies held synthetic to keep the program linear; the real frequencies from evaluation are never fed back to the optimizer.
- **Suffix overlap.** The double-counting penalty over-estimates the overlap, so the optimizer can discard an overlapping suffix pair that the true optimum would keep.
- **Noise.** The uncertainty penalty curbs noisy candidates but does not remove them, and k_σ is an arbitrary one standard deviation.

A direct search could instead optimize the true objective in full context. Simulated annealing or a genetic algorithm would work. This drops the simplifications, but gives up the optimality guarantee and costs far more compute.

Two assumptions, unlike the optimizer's crude shortcuts, can push the measured number too high. The largest is the frequency assumption. When an abbreviation is scored, its word-frequency feature is set to the value of the word it replaces, not of the abbreviation itself. This assumes the typist recalls the mapping at once, as if it were learned as well as any common word. In reality a typist pauses to recall a mapping, more so for rare or odd entries. The measured speedups are therefore likely too high. Modeling this needs behavioral data and is left for future work.

The second is the trigger key's hand assignment. It inherits the spacebar's ambidextrous assignment, since the model was trained with the spacebar typed by either thumb. A dedicated trigger bound to one thumb could cost slightly more than a space, so the reported speedup is optimistic on this count. A fixed-hand assignment was rejected because it would place the key outside the model's training distribution.

The keys' slots are a further fixed assumption, but a limitation rather than a bias. They are placed by hand at plausible slots, not optimized. The repeat key adds only 0.20 percentage points, so its placement barely affects the result. It could be dropped for a simpler one-key device at almost no cost. The trigger key is different. It carries almost the whole speedup, so its placement and cost matter more. The trigger placement here is plausible but unoptimized, and finding a better one is left

open. The current trigger key placement is not hypothetical: split-spacebar keyboards already put a second thumb key in this spot.

Intuitiveness is the softest part. There is no objective measure of how memorable an abbreviation is, so the heuristic encodes one person's judgment. An LLM scorer was tried but did not work: its scores were too inconsistent.

The dictionary is also unsystematic. Each abbreviation is chosen on its own, so related words need not get related forms. `"panda"` may shorten to `"pa"`, yet `"pandas"` need not become `"pas"`. The typist has no rule to generalize from and must memorize each entry separately. Memorizing many unrelated mappings is itself costly. The intuitiveness heuristic scores each abbreviation alone, so it misses this set-wide cost. The burden grows with dictionary size. A systematic scheme, with fixed rules from words to abbreviations, may be easier to learn, but would give up per-word speed tuning. A speed model would still help there: it could rank which rules save the most time, or reserve unsystematic, speed-optimized forms for the few highest-frequency words, where the gain is largest.

Whole words outperform suffixes by far, 12.33% against 3.18%. Which points past single words to multi-word phrases. A phrase like "red panda" could collapse to `rp`, or `btw` to "by the way". Extending the candidate pool to phrases is left for future work.

6.3.5 Conclusion

Trained IKI models can drive abbreviation-dictionary design end to end. Judged by a separate model, the speedup runs from about 5% at 10 entries to 17% at 612 entries, an upper bound that assumes instant recall of each mapping. Whole words carry most of it; suffixes add under a point. Size and required intuitiveness set where a dictionary lands in that range.

6.4 Is Dvorak faster than QWERTY?

6.4.1 Introduction

Dvorak claimed the layout minimizes finger movement to increase typing speed compared to QWERTY [5]. The actual benefit has been debated [14], [15]. A 1956 study retrained typists and found Dvorak no faster than QWERTY [3]. The subjects were already biased to know QWERTY. The study used 10 subjects per group. The small sample size leaves the question open.

Kinkead estimated layout speed using fixed movement costs [7]. The model is very simple. It does not explain much of what affects typing speed. The model suffers from QWERTY practice bias.

This study applies the trained LightGBM typing model to measure the speed difference.

6.4.2 Methodology

This study evaluates the Dvorak layout using the trained LightGBM model. LightGBM is selected for two reasons. It is the only model with no statistically significant layout bias (Section 4.2), so the result depends least on the bias correction. It also achieves the best zero-shot Dvorak accuracy (Table 11).

The model scores 25 000 sentences from the combined Reddit and Wikipedia corpus (Section 3.1.2). For each keystroke it predicts the normalized inter-keystroke interval. A lower value indicates faster typing. The same sentences are scored on both QWERTY and Dvorak. The difference is converted to a percentage speedup using the dataset's mean IKI standard deviation (50.4 ms) and mean IKI (110.5 ms).

6.4.3 Results

LightGBM predicts a mean normalized interval of 0.001 for QWERTY and −0.013 for Dvorak. Dvorak is predicted to be faster than QWERTY by a raw advantage of 0.014 standard deviations.

The raw predictions must be adjusted for layout bias. LightGBM exhibits a known bias shift of +0.0015 (Section 4.2). The model slightly underestimates Dvorak speed. Adjusting for this bias yields an advantage of 0.0158 standard deviations, equivalent to a 0.7% typing speedup for Dvorak.

Applying the uncertainty of the bias shift yields an adjusted Dvorak speedup interval ranging from 0.5% to 1.0% ($p < 0.001$). The entire interval is positive, which indicates a motor advantage for Dvorak.

6.4.4 Discussion

This interval understates the true uncertainty. The underlying confidence intervals resample sentences, not participants, so they miss between-typist variation (Section 5.3). The 0.7% is therefore directional rather than exact.

The analysis would be more precise if the model were fine-tuned on Dvorak typing data.

6.4.5 Conclusion

Data-driven modeling indicates a 0.7% typing speedup for Dvorak over QWERTY. The estimate is positive across the bias-shift interval, but the intervals omit between-typist variation, so the 0.7% is an indication rather than a settled result.

7 Conclusion

Machine learning models predict typing speed from keystroke data. Nonlinear models outperform linear ones. Including future context improves QWERTY accuracy. This confirms typing is anticipatory. The models also generalize from QWERTY to Dvorak, the first zero-shot evaluation of a typing-speed model on a different layout. LightGBM transfers best. As the deployed ensemble, it reaches the lowest Dvorak MAE (0.588) and the least bias shift. This makes it the most trustworthy across layouts. MLP Main scores best on QWERTY (0.566). But its edge vanishes on Dvorak. There the near-raw-coordinate MLP DL nearly matches it, a sign MLP Main overfits.

Whether the model can guide layout optimization remains unclear. Its bias on layouts beyond QWERTY and Dvorak is unmeasured. LightGBM is the least biased model, but also the slowest. Scoring the many candidate layouts optimization requires is computationally impractical.

The models quantify the speed benefits of typing system optimizations. One-shot shift, on the displaced semicolon slot, saves 18.5 ms per isolated capital, a 0.71% speedup, or about 1.1% if the benefit extends to sentence-initial capitals. A repeat key in that slot gives a 0.26% gain. Abbreviation dictionaries increase speed by up to 12.99% for a 160-entry dictionary, an estimate that assumes instant recall of each mapping. After adjusting for layout bias, Dvorak is 0.7% faster than QWERTY.

Two directions would improve the models. The first is data: only QWERTY and Dvorak have keystroke records. More layouts would lower the MAE, shrink the bias shift, and tighten the confidence intervals. The second is architecture: simpler window models and fewer features would likely reduce the overfitting that MLP Main shows.

A Appendix

A.1 Keyboard Layouts

To evaluate cross-layout generalization, models trained on QWERTY are tested on alternative layouts. Visual representations of these layouts are provided for reference.

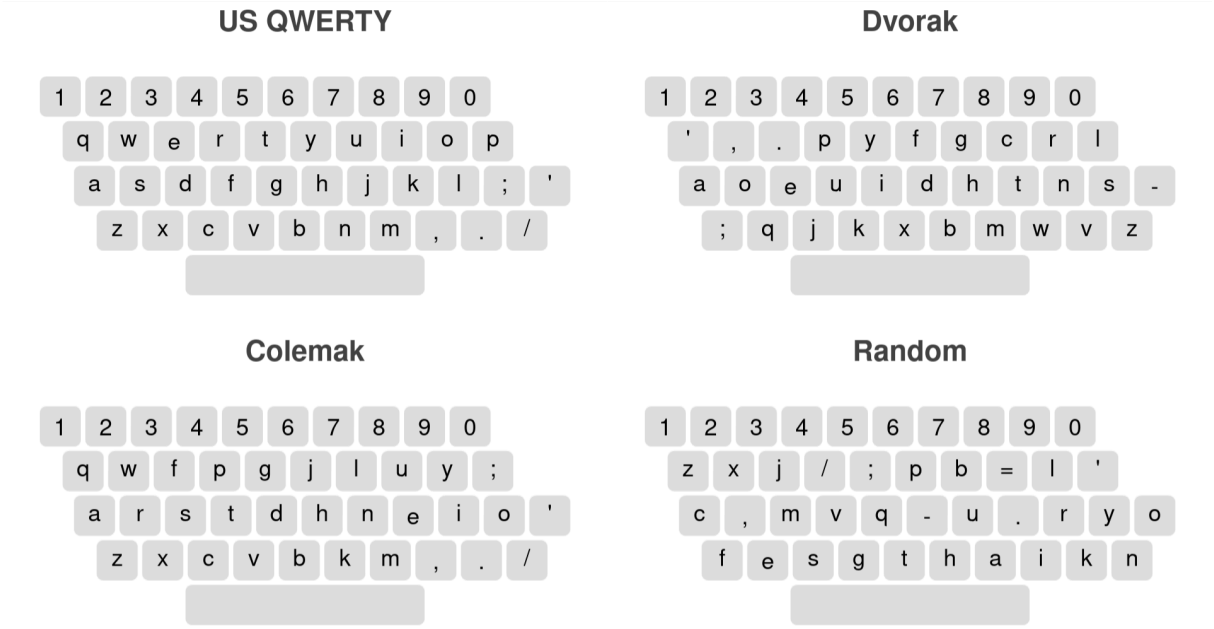

Figure 6: The four layouts evaluated in this study: US QWERTY (top left), Dvorak (top right), Colemak (bottom left), and a procedurally generated Random layout (bottom right) used to test extreme out-of-distribution performance.

A.2 SHAP feature importance

SHAP values were computed during feature selection (see the LGBM and MLP subsections). For each model, two rankings are shown: one on the full candidate set and one on the reduced set after ablation. A `random_noise` feature (standard normal, no signal) is included in every run as an objective cutoff — features ranking below it are uninformative.

A.2.1 LGBM, all features

Rank	Feature	SHAP
1	`bigram_frequency`	0.22321
2	`word_frequency`	0.17523
3	`finger`	0.10542
4	`same_finger`	0.08113
5	`unigram_frequency`	0.07365
6	`same_hand`	0.06400
7	`move_dist`	0.05593
8	`repetition`	0.03150
9	`same_finger_skipgram`	0.02739
10	`move_sin`	0.02359
11	`x`	0.02280
12	`y`	0.02278
13	`word_relative_pos`	0.02256
14	`word_length`	0.02027
15	`move_cos`	0.01924

Rank	Feature	SHAP
16	is_syllable_start	0.01722
17	double_row_jump	0.01553
18	out_roll	0.01513
19	sequence_pos	0.01262
20	in_roll	0.01208
21	shift	0.01000
22	skipgram_repetition	0.00962
23	word_index	0.00907
24	is_syllable_end	0.00788
25	redirects	0.00619
26	hand	0.00617
27	sequence_length	0.00400
28	sequence_relative_pos	0.00365
29	finger_type	0.00244
30	is_word_end	0.00215
31	**random_noise**	**0.00178**
32	in_triroll	0.00074
33	is_word_start	0.00066
34	same_finger_trigram	0.00054
35	out_triroll	0.00053
36	scissors	0.00039
37	is_pad	0.00000

Table 23: LGBM SHAP feature importance on all 36 candidate features (plus the `random_noise` control). Computed via `shap.TreeExplainer` on 50,000 validation samples. Validation MAE: 0.5852 [95% CI: 0.5833, 0.5872].

A.2.2 LGBM, reduced 23 features

Rank	Feature	SHAP
1	bigram_frequency	0.20960
2	word_frequency	0.17620
3	finger	0.12619
4	same_finger	0.08229
5	move_dist	0.07721
6	same_hand	0.06852
7	y	0.03581
8	repetition	0.03349
9	x	0.03048
10	same_finger_skipgram	0.02787
11	move_sin	0.02783
12	word_relative_pos	0.02548

Rank	Feature	SHAP
13	move_cos	0.02016
14	word_length	0.01961
15	shift	0.01825
16	word_index	0.01799
17	double_row_jump	0.01752
18	sequence_pos	0.01618
19	out_roll	0.01494
20	skipgram_repetition	0.01161
21	in_roll	0.01079
22	hand	0.00985
23	redirects	0.00549
24	**random_noise**	**0.00287**

Table 24: LGBM SHAP feature importance after ablation, on the final 23-feature set. Validation MAE: 0.5771 [95% CI: 0.5751, 0.5790].

A.2.3 MLP, all features

Rank	Feature	SHAP
1	word_frequency	0.20461
2	bigram_frequency	0.18802
3	move_dist	0.09535
4	shift	0.09126
5	move_sin	0.08026
6	same_hand	0.07798
7	move_cos	0.06875
8	same_finger	0.06404
9	x	0.06051
10	unigram_frequency	0.05010
11	is_syllable_start	0.04729
12	word_length	0.04547
13	repetition	0.04444
14	is_word_start	0.03962
15	is_syllable_end	0.03879
16	hand_0	0.03783
17	in_roll	0.03695
18	finger_6	0.03592
19	word_index	0.03581
20	hand_1	0.03486
21	is_word_end	0.03305
22	finger_type_3	0.03291
23	hand_2	0.03219

Rank	Feature	SHAP
24	out_roll	0.03137
25	same_finger_skipgram	0.02964
26	y	0.02960
27	double_row_jump	0.02921
28	finger_4	0.02910
29	skipgram_repetition	0.02894
30	finger_type_4	0.02726
31	finger_2	0.02635
32	finger_type_2	0.02443
33	finger_3	0.02394
34	word_relative_pos	0.02382
35	same_finger_trigram	0.02164
36	sequence_relative_pos	0.02028
37	finger_9	0.02007
38	finger_type_0	0.01840
39	redirects	0.01838
40	in_triroll	0.01818
41	out_triroll	0.01439
42	finger_8	0.01430
43	finger_type_1	0.01426
44	finger_0	0.01284
45	sequence_length	0.01242
46	sequence_pos	0.01225
47	finger_7	0.00985
48	finger_1	0.00897
49	**random_noise**	**0.00691**
50	scissors	0.00641
51	is_pad	0.00610
52	finger_5	0.00000

Table 25: MLP SHAP feature importance on all 51 candidate features (plus the `random_noise` control). Computed via `shap.DeepExplainer` with 300 background and 700 test samples. Validation MAE: 0.5758 [95% CI: 0.5737, 0.5778].

A.2.4 MLP, reduced 35 features

Rank	Feature	SHAP
1	word_frequency	0.21279
2	bigram_frequency	0.18681
3	move_dist	0.11895
4	shift	0.09630
5	move_sin	0.08676

Rank	Feature	SHAP
6	move_cos	0.07469
7	same_hand	0.07331
8	same_finger	0.06606
9	finger_4	0.05777
10	x	0.05530
11	finger_6	0.05373
12	word_length	0.05194
13	hand_0	0.04867
14	hand_2	0.04809
15	is_syllable_start	0.04415
16	hand_1	0.04389
17	finger_3	0.04198
18	in_roll	0.04178
19	repetition	0.03776
20	y	0.03653
21	is_syllable_end	0.03647
22	word_relative_pos	0.03643
23	out_roll	0.03216
24	finger_2	0.03169
25	same_finger_skipgram	0.03161
26	skipgram_repetition	0.03066
27	double_row_jump	0.02959
28	finger_9	0.02266
29	redirects	0.02070
30	finger_0	0.01918
31	finger_8	0.01807
32	finger_1	0.01171
33	finger_7	0.01108
34	sequence_pos	0.00967
35	**random_noise**	**0.00964**
36	finger_5	0.00000

Table 26: MLP SHAP feature importance on the reduced 35-feature set. Validation MAE: 0.5714 [95% CI: 0.5695, 0.5734].

A.3 Hyperparameter Optimization Details

This section details the search spaces and final hyperparameter configurations.

A.3.1 Linear Regression

Search space (per trial):

- `penalty`: `l1`, `l2`, or `elasticnet`
- `alpha`: 10^{-5} to 10^{-1}, log-scaled

- `l1_ratio`: 0.0 to 1.0 (only if `penalty="elasticnet"`)
- `learning_rate`: `invscaling` or `adaptive`
- `eta0`: 10^{-3} to 10^{-1}, log-scaled

Best hyperparameters:

```
{
    "w_back": 1, "w_ahead": 1,
    "penalty": "l2",
    "alpha": 0.01216,
    "learning_rate": "adaptive",
    "eta0": 0.00171,
}
```

A.3.2 LGBM

Search space (per trial):

- `learning_rate`: 0.01 to 0.2, log-scaled
- `num_leaves`: 15 to 255
- `max_depth`: −1 (unlimited) to 15
- `min_child_samples`: 10 to 100
- `subsample`: 0.5 to 1.0
- `colsample_bytree`: 0.5 to 1.0
- `reg_alpha`: 10^{-8} to 10.0, log-scaled
- `reg_lambda`: 10^{-8} to 10.0, log-scaled

Best hyperparameters:

```
{
    "w_back": 3, "w_ahead": 1,
    "learning_rate": 0.08865,
    "num_leaves": 73,
    "max_depth": 13,
    "min_child_samples": 41,
    "subsample": 0.7485,
    "colsample_bytree": 0.8856,
    "reg_alpha": 0.13770,
    "reg_lambda": 0.03024,
}
```

A.3.3 MLP Main

Search space (per trial):

- `n_layers`: 1 to 4
- `hidden_dim`: 64, 128, 256, 512, or 1024
- `dropout`: 0.0 to 0.5
- `activation`: `ReLU`, `GELU`, or `SiLU`
- `lr`: 10^{-4} to 10^{-2}, log-scaled
- `weight_decay`: 10^{-6} to 10^{-2}, log-scaled
- `batch_size`: 8192, 16384, 32768, or 65536

Best hyperparameters:

```
{
    "w_back": 2, "w_ahead": 1,
    "n_layers": 2,
    "hidden_dim": 1024,
    "dropout": 0.0800,
```

```
    "activation": "GELU",
    "lr": 0.001428,
    "weight_decay": 0.001833,
    "batch_size": 32768,
}
```

A.3.4 MLP DL

Search space (per trial): Same as MLP Main, except `n_layers`: 1 to 5.

Best hyperparameters:

```
{
    "w_back": 2, "w_ahead": 1,
    "n_layers": 3,
    "hidden_dim": 256,
    "dropout": 0.0950,
    "activation": "GELU",
    "lr": 0.002053,
    "weight_decay": 0.000344,
    "batch_size": 16384,
}
```

A.3.5 MLP Linguistic

Search space (per trial): Same as MLP Main.

Best hyperparameters:

```
{
    "w_back": 2, "w_ahead": 1,
    "n_layers": 2,
    "hidden_dim": 1024,
    "dropout": 0.3861,
    "activation": "SiLU",
    "lr": 0.002592,
    "weight_decay": 0.000824,
    "batch_size": 8192,
}
```

A.4 Selected Features

This section details the specific features selected for each model.

A.4.1 Linear Regression

```
LINREG_FEATURES = [
    "is_pad", "move_dist", "move_sin", "move_cos", "x", "y", "shift",
    "unigram_frequency", "bigram_frequency", "word_frequency",
    "same_hand", "same_finger", "same_finger_trigram",
    "repetition", "skipgram_repetition", "same_finger_skipgram",
    "in_roll", "out_roll", "in_triroll", "out_triroll",
    "redirects", "double_row_jump", "scissors",
    "word_index", "word_length", "word_relative_pos",
    "is_word_start", "is_word_end", "is_syllable_start", "is_syllable_end",
    "finger_0", "finger_1", "finger_3", "finger_6", "finger_8", "finger_9",
    "finger_type_0", "finger_type_1", "finger_type_2", "finger_type_3",
    "hand_0",
]
```

A.4.2 LGBM

```
LGBM_FEATURES = [
    'move_dist', 'move_cos', 'move_sin', 'x', 'y', 'shift',
    'bigram_frequency', 'word_frequency',
    'same_hand', 'same_finger',
    'repetition', 'skipgram_repetition', 'same_finger_skipgram',
    'in_roll', 'out_roll', 'redirects', 'double_row_jump',
    'sequence_pos', 'word_index', 'word_length', 'word_relative_pos',
    'finger', 'hand'
]
```

A.4.3 MLP Main

```
MLP_MAIN_FEATURES = [
    "move_dist", "move_cos", "move_sin", "x", "y", "shift",
    "bigram_frequency", "word_frequency",
    "repetition", "skipgram_repetition", "same_finger_skipgram",
    "same_finger", "same_hand",
    "in_roll", "out_roll", "redirects", "double_row_jump",
    "sequence_pos", "word_length", "word_relative_pos",
    "is_syllable_start", "is_syllable_end",
    "finger_0", "finger_1", "finger_2", "finger_3", "finger_4",
    "finger_5", "finger_6", "finger_7", "finger_8", "finger_9",
    "hand_0", "hand_1", "hand_2",
]
```

A.4.4 MLP Linguistic

```
MLP_LINGUISTIC_FEATURES = [
    "bigram_frequency", "word_frequency",
    "sequence_pos", "word_length", "word_relative_pos",
    "is_syllable_start", "is_syllable_end",
]
```

A.4.5 MLP Deep Learning

```
MLP_DL_FEATURES = [
    "x", "y", "shift", "hand",
    "bigram_frequency", "word_frequency",
    "sequence_pos", "word_length", "word_relative_pos",
    "is_syllable_start", "is_syllable_end",
]
```

Bibliography

[1] V. Dhakal, A. M. Feit, P. O. Kristensson, and A. Oulasvirta, *Observations on Typing from 136 Million Keystrokes*. ACM, 2018.

[2] A. M. Anderson, G. A. Mirka, S. M. B. Joines, and D. B. Kaber, "Analysis of Alternative Keyboards Using Learning Curves," 2009.

[3] E. P. Strong, *A Comparative Experiment in Simplified Keyboard Retraining and Standard Keyboard Supplementary Training*. U.S. General Services Administration, 1956. [Online]. Available: https://archive.org/details/AComparativeExperimentInSimplifiedKeyboardRetraining AndStandardKeyboardSupplementaryTraining/mode/2up

[4] E. A. Williams, M. Warburton, M. Krzywinski, and F. Mushtaq, "The Typability Index: A tool for measuring and controlling for typing difficulty in text stimuli," 2026.

[5] A. Dvorak, N. L. Merrick, W. L. Dealey, and G. C. Ford, *Typewriting Behavior: Psychology Applied to Teaching and Learning Typewriting*. American Book Company, 1936.

[6] M. Krzywinski, "CARPALX — keyboard layout optimizer." [Online]. Available: https://mkweb.bcgsc.ca/carpalx/

[7] R. Kinkead, "Typing Speed, Keying Rates, and Optimal Keyboard Layouts," 1975.

[8] Y. Hiraga, Y. Ono, and H. Yamada, *An analysis of the standard English keyboard.* in COLING '80: Proceedings of the 8th conference on Computational linguistics. Association for Computational Linguistics, 1980.

[9] A. İşeri and M. Ekşioğlu, "Estimation of digraph costs for keyboard layout optimization," 2015.

[10] T. A. Salthouse, "Perceptual, cognitive, and motoric aspects of transcription typing," 1986.

[11] J. Baumgartner, S. Zannettou, B. Keegan, M. Squire, and J. Blackburn, "The Pushshift Reddit Dataset," 2020.

[12] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel Mixture Models," 2016.

[13] G. C. Vanderheiden, "Computers can play a dual role for disabled individuals," 1982. [Online]. Available: https://archive.org/details/byte-magazine-1982-09

[14] J. Noyes, "The QWERTY keyboard: A review," 1983.

[15] S. J. Liebowitz and S. E. Margolis, "The Fable of the Keys," 1990. [Online]. Available: https://www.jstor.org/stable/725385